Application Security Engineering — Study Plan

Independent Self-Study — Kyle Miskell — kmiskell@protonmail.com

0	I	II	III	IV	V	VI	VII	VIII	IX
Pre-Study	AppSec Intro	Web & Network	Core Vulns	Testing & Tooling	Cloud Security	Code Review	Sec+ Cert	Interview Prep	First Days

Contents

What Is Application Security?

Application security — AppSec — is the discipline of finding, fixing, and preventing security vulnerabilities in software. Where most of software engineering asks *does this work?*, application security asks *can this be broken?* The question sounds simple. The implications are not.

Software has an attack surface: every input field, every API endpoint, every authentication flow, every dependency in your requirements.txt is a potential entry point for an adversary. AppSec engineers systematically map that surface, probe it for weaknesses, and work with development teams to close them — before attackers find them first.

The discipline spans the entire software development lifecycle. In the design phase, it means threat modeling — drawing data flows and asking what happens when an attacker controls each input. During development, it means secure code review — reading code the way an attacker reads it, looking for injection points, broken authentication, insecure data handling. In testing, it means static analysis (SAST) that reads source code for vulnerability patterns, dynamic analysis (DAST) that probes a running application, and manual penetration testing that combines both with adversarial creativity.

The OWASP Top 10 is the field's shared vocabulary for the most critical web application security risks. SQL injection, broken access control, cross-site scripting, security misconfigurations, insecure deserialization, and other key risks. These are not abstract academic concerns — they are the specific vulnerability classes that appear in real breaches against real companies, year after year. AppSec engineers know them the way a surgeon knows anatomy: not as a list to memorize, but as a framework for understanding how systems fail.

This plan targets web application security specifically — the attack surface that lives at the HTTP layer, in APIs, in authentication flows, in the browser.

Why AppSec Engineering

The Case for This Path

This plan is a deliberate pivot, not a hedge. You spent five years building the attack surface that AppSec engineers protect. You know FastAPI microservices, React frontends, AWS infrastructure, Docker containers, poorly written legacy PHP codebases — the exact technology stack that AppSec interviews test your ability to reason about adversarially. No other security specialization maps as directly to a full-stack development background.

Your Structural Advantage

Most AppSec engineers come from one of two directions: former developers who learned security, or security professionals who learned development. You are in the first category — and the first category produces better AppSec engineers, because you understand not just what the vulnerability is but why developers write it. The code patterns that lead to SQL injection, authentication bypasses, and CORS misconfigurations are patterns that developers encounter every day. Recognizing them as vulnerabilities — rather than just implementation choices — is the shift this plan builds. That background is not a liability to overcome. It is the primary qualification.

Specific transferable skills from your resume:

- **FastAPI microservices** — REST API security is the dominant AppSec domain in 2026. Hands-on API security knowledge built throughout this plan means arriving on day one prepared for the work.
- **Python** — The primary scripting language for AppSec tooling. Custom Semgrep rules, vulnerability scanning automation, security utilities.
- **React frontends** — Client-side vulnerabilities (XSS, CSRF, insecure storage) require understanding how the browser executes JavaScript. React experience provides a strong foundation for understanding client-side vulnerability patterns.
- **AWS, Docker, Kubernetes** — Cloud and container security are AppSec-adjacent and increasingly part of the role. Your infrastructure knowledge is a differentiator.
- **Legacy PHP, raw SQL, and NoSQL** — You have worked in poorly written legacy PHP, hand-rolled MySQL, and DynamoDB — exactly where SQL injection, IDOR, and NoSQL injection live. You recognize the vulnerable pattern and know how to fix it.
- **Architecture & domain modeling** — You designed microservices and domain-driven services with deliberate separation of concerns and trust boundaries. Threat modeling is that same structural thinking turned toward attack: mapping data flows and asking where trust is assumed.
- **Testing discipline** — You wrote unit, integration, and regression tests. AppSec testing follows the same deterministic logic: inputs, expected behavior, failure modes.

No LeetCode

AppSec interviews test security knowledge — OWASP concepts, secure code review, threat modeling, vulnerability identification. Not dynamic programming. Not graph traversal. The technical interview involves reading vulnerable code and explaining what is wrong with it — something five years of full-stack experience of

building code makes you structurally prepared for. The relief this should produce as part of a break from never-ending leetcode grinding, contrived software engineering interviews, and poor communication during the full software eng interview lifecycle is real and substantial.

Why This Moment

The case for entering application security in 2026 rests on concrete numbers, not hype. Here is what the data actually shows.

The breach problem is not going away.

IBM's 2025 Cost of a Data Breach Report — the most cited annual benchmark in the industry, now in its 20th year — found that the average U.S. cost of a data breach reached a record $10.22 million in 2025, even as the global average fell slightly to $4.44 million driven by faster AI-assisted detection. The U.S. number is the highest ever recorded. Nearly two-thirds of the 600 organizations IBM studied said they were still recovering from their breach at the time of the report. Verizon's 2025 Data Breach Investigations Report found that exploitation of vulnerabilities — the class of attack that AppSec engineers specifically exist to prevent — initiated 20% of all breaches studied.

The application layer is where the majority of those vulnerabilities live. More than 45% of data breaches in 2024 involved at least one application-layer vulnerability. The number of web applications worldwide exceeded 1.8 billion in 2024, increasing the attack surface faster than the security workforce is growing. Over 60% of Fortune 500 companies now integrate application security testing into their software development lifecycle — which means nearly 40% still do not, and most of those companies are actively looking for people to fix that.

The workforce gap is structural.

ISC2's 2025 Workforce Study estimated 4.8 million unfilled cybersecurity roles globally. The U.S. Bureau of Labor Statistics projects 29% job growth in information security analysis from 2024 to 2034 — nearly ten times the average growth rate across all occupations. U.S. employers posted over 514,000 cybersecurity job openings in the past twelve months, up 12% from the prior year. For application security specifically, Mordor Intelligence reports a 15% annual shortfall of AppSec engineers in the United States through 2026 — demand growing faster than the supply of qualified people to fill it.

The shortage is not evenly distributed. ISC2 noted in 2025 that skills shortages now outweigh headcount shortages as the primary constraint — meaning companies are not just short on bodies, they are short on people who can actually do the work. AI is automating the lowest complexity tasks (Tier 1 alert triage, routine vulnerability scanning), which is compressing demand for those roles. What it is not automating is the judgment-intensive work: finding business logic flaws that automated scanners cannot detect, threat modeling a new microservice architecture, explaining a critical finding to a VP of Engineering in terms that result in a fix. That work requires a human who can read code, understand systems, and think adversarially. That is the work this plan prepares you for.

The AppSec market is expanding fast.

The global application security market was valued at $13.68 billion in 2024 and is projected to reach $53.25 billion by 2033, growing at 16.3% annually. The application security testing market specifically is growing at 26.7% annually — from $1.83 billion in 2025 to a projected $7.60 billion by 2031. Salary data from 2026 job postings shows Application Security Engineers earning $130,000–$180,000 at mid-level, driven by what one salary analysis describes as 'strong demand due to OWASP Top 10 vulnerabilities and insecure APIs.' Product Security Engineer postings grew 12.08% year over year in 2025, outpacing most other security roles in posting

growth.

The structural argument: IBM’s data shows that organizations with severe security staffing shortages face $1.76 million higher breach costs than those that are adequately staffed. Companies are not hiring AppSec engineers because it is a nice-to-have. They are hiring because the cost of not having them is measurable, large, and growing.

Target Role: Application Security Engineer

What It Is — Conceptually

Web application development and application security share the same foundation: HTTP, APIs, authentication flows, browser security models, database queries. Every endpoint in a web application is an AppSec concern. Every authentication flow is a potential bypass. Every React component that renders user input is an XSS candidate. Developers who learn AppSec understand the systems being defended — they are adding an adversarial lens to knowledge they already hold.

The shift is perspective. Where a developer asks *does this feature work correctly?*, an AppSec engineer asks *what happens when an attacker controls this input?* The underlying knowledge of the system is identical. The adversarial lens is the discipline.

What It Is — In Practice

- **Secure code review** (analogous to: pull request review, but adversarial) — Reading code to identify vulnerability patterns: SQL queries built by string concatenation, tokens stored in localStorage, endpoints missing authorization checks, user data rendered without sanitization.
- **Penetration testing / DAST** (analogous to: integration testing, but adversarial) — Running Burp Suite against a staging environment, intercepting HTTP requests, modifying parameters, observing whether the application behaves securely.
- **SAST tooling** (analogous to: adding a linter to CI/CD) — Integrating Semgrep into the pipeline. Writing custom rules that catch company-specific vulnerability patterns. Triaging scan results: true positives from false positives.
- **Threat modeling** (analogous to: architecture review, but adversarial) — Given a proposed system design, enumerate the ways it could be attacked. STRIDE: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.
- **Vulnerability management** (analogous to: bug triage) — Prioritizing findings by severity and business impact. Writing clear reports developers can act on. Tracking remediation. Verifying fixes.

Day-to-Day Responsibilities

- Secure code review on high-risk PRs and new features — auth, payments, file upload, API endpoints
- Running and maintaining SAST tooling in CI/CD — Semgrep rules, false positive triage, new rule development
- Manual penetration testing of web applications and APIs — Burp Suite, methodology-driven, OWASP-aligned
- Threat modeling new system designs — STRIDE analysis, data flow diagrams, risk documentation
- Vulnerability reporting and remediation tracking — clear writeups, severity ratings, developer liaison
- Security training and developer education — lunch-and-learns, secure coding guidelines, champions programs
- Dependency and supply chain security — SCA scanning, CVE monitoring, patch prioritization

Compensation — Summer 2026

Level	Base Salary	Notes
Entry / first role	$90K–$130K	Realistic with strong portfolio; varies by company size
Mid-level (2–4 yrs)	$130K–$180K	Glassdoor avg $164K; remote-first roles higher
Senior (4+ yrs)	$170K–$240K	Deep domain expertise, mentoring, architecture involvement
Top tech (FAANG+)	$250K–$400K+	Total comp incl. equity; not a realistic first role target

Philadelphia & Remote Market

AppSec has strong remote availability — roughly 70–75% of postings offer remote or hybrid flexibility. Philadelphia-based remote work means accessing national salary bands at Philadelphia cost of living. Local Philly demand is moderate but real, anchored by healthcare, financial services, and enterprise tech.

Company Type	Examples	Role Fit
Healthtech	Penn Medicine, Jefferson Health, Abridge (remote)	Strong. PHI/HIPAA requirements drive AppSec demand.
Financial services	Vanguard (Malvern), JPMorgan (Wilmington)	Strong. PCI compliance, high-value targets, established AppSec programs.
Fintech / Crypto	Remote-first nationally; Coinbase, Kraken, Block	Best fit. Finance/crypto pay premium. Fast hiring on demonstrated skill.
Enterprise (Philly)	Comcast, SAP Concur, Independence Blue Cross	Good. Stable. Full-stack background relevant to AppSec integration work.
Legal AI / LegalTech	Thomson Reuters/Casetext, Gap International (Philly)	Good. Document handling, PII, sensitive data — AppSec-heavy by necessity.

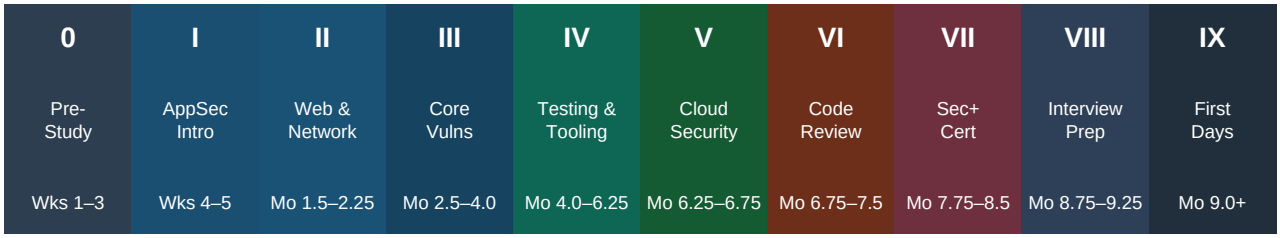
Start applying at Month 4 for AI-native startups and security-conscious tech companies. Target Philadelphia legal AI and healthtech in parallel — these companies hire locally, are less saturated with remote applicants, and a full-stack background is immediately relevant.

Plan Schedule Overview

This plan is calibrated for 7–9 months at 21 study hours per week. 7 months is the optimistic case; 9 months is the outer bound; 8 months is the realistic midpoint. The hardest phase is not technical — it is mental: shifting from builder to adversarial thinker.

Day Type	Days/Week	Hours	Notes
Long Study	3	7 hrs	Reading, labs, implementation, tool practice, note-card review, writeups
TOTAL STUDY	**3**	**21 hrs**	
Work (Gigs)	2	7 hrs	Income — Two 3–4 hour shifts
Day Off	2	—	No study, no work. Required for retention & sanity.

Phase Timeline



Timeline: 7 months is the optimistic case; 9 months is the outer bound; 8 months is the realistic midpoint. The first 2–3 weeks are pre-study before security content begins. Start applying at Month 4 for startups and AppSec-adjacent roles. Security+ lands on your resume at Month 8.5, strengthening the main application push that follows. On the two-year employment gap: ‘I have been in a deliberate full-time transition into application security since June 2026. This plan is the evidence.’

Study Methodology

The methods below apply to every phase. Following them consistently is the difference between completing labs and actually owning the material.

Human Mentorship: MentorCruise

Self-study hits a ceiling when a vulnerability class just will not click, or when you need someone to tell you whether your code review findings are the right findings. A mentor who ships AppSec work professionally can review this plan, calibrate your lab work, and run mock code review sessions. MentorCruise (mentorcruise.com) is the platform.

- **What to look for:** Production AppSec engineering background. Keywords: application security, secure code review, penetration testing, OWASP, Burp Suite, AppSec program building. Avoid pure red-team/CTF backgrounds.
- **Suggested usage:** One session upfront to review this plan. Monthly check-ins at phase transitions. Regular mock interviews (code review and behavioral) from mid-plan onward — running well before your first real interview. Async questions for anything that blocks progress.
- **Cost:** $100–$150/month. 7-day trial lets you test fit. Highest-ROI expenditure in the plan.
- **When to start:** Before Phase I — and make the very first session a hard review of this plan itself: is it a realistic route into an entry-level AppSec role, and what should change? Ask for concrete edits — what to add, cut, or update.

Local & Regional Events

Studying is not only reading and labs — attending local and regional security events is part of the work, both to rebuild the sense of being part of the industry after heads-down solo study and because referrals come from people who have met you. A few worth knowing: the OWASP Philadelphia chapter (the most directly relevant group), Philly 2600 (the lowest-pressure entry point), and OWASP Global AppSec in nearby Washington DC. Start showing up early — even one finished PortSwigger lab is enough to bring and talk about. See the Appendix section “Philadelphia AppSec Community” for the full list of groups and conferences.

Note-Taking: Notes → Note Cards

Two outputs for every chapter or lab section: working notes during study, note cards derived from them afterward. Notes are for comprehension; note cards are for retention.

- **During study:** Detailed working notes in your own words. For security concepts: what the vulnerability is, why the code is exploitable, how an attacker uses it, how to fix it. Questions you cannot answer yet.
- **After each section:** Extract note cards. One concept per card. Front: vulnerability name. Back: what it is, how to identify it in code, how to fix it. If you cannot explain the fix without looking, you do not own the concept.
- **Chapters marked Skip Note-Taking:** One-paragraph summary only. Everything in Phases II–V gets full note cards — this is the interview material.

Portfolio: Writeups & Project Site

AppSec portfolios are built differently from software engineering portfolios. What distinguishes an AppSec candidate is documented offensive work: vulnerability writeups, lab solutions, CTF walkthroughs, threat model documents. Security hiring managers look for evidence of real thinking — someone who can find a vulnerability, understand why it exists, exploit it, and explain the fix. A well-written writeup demonstrates all four in a way a resume line never can.

- **Writeup format:** For every major lab or project: what the vulnerability was, how you identified it, the exploitation proof-of-concept, how to remediate it, what you look for in a code review to catch it. 800–1,200 words. Publish polished versions on LinkedIn and a personal site.
- **Target:** at least two published writeups before you begin applying, building to 4–6 before the main application push. Quality over quantity.
- **GitHub:** Commit from Day 1 — the very first commit is this study plan and its build script. Keep one repository for the entire cybersecurity journey: notes, tooling-phase scripts, custom Semgrep rules, automation tools, and lab artifacts, building a continuous commit history from the start. Every README explains the methodology and what the artifact demonstrates — not just usage instructions.

AppSec: Potential Pitfalls & Risk Assessment

AI Replacing AppSec Engineers — Real but Limited

AI-assisted SAST tools (GitHub Copilot Autofix, Snyk Code, Semgrep with AI triage) are improving and will automate more routine vulnerability scanning over time. The roles being automated are the lowest-end scan-and-report functions, not the manual code review, threat modeling, and developer consultation that constitutes senior AppSec work. Arriving with a portfolio of demonstrated manual security work positions you above the automation floor from day one.

Entry-Level Hiring Compression

AI tools are handling some work that junior security analysts used to do. The mitigation is the same as it has always been in security hiring: arrive with a portfolio of real work rather than a list of certifications. Someone who can walk through a stored XSS chain and connect it to a real bypass they identified during code review is not competing with the Security+ certificate cohort.

The Certification Trap

Sec+ gets you past ATS filters and signals baseline literacy. It does not substitute for demonstrated hands-on skill in interviews. The plan treats Sec+ as a necessary filter-passer, not the credential. The portfolio is the credential.

Moderate Philly Market

Philadelphia's AppSec market is real but not as deep as New York, DC, or San Francisco. The mitigation is remote — 70–75% of AppSec postings offer remote or hybrid. Philadelphia is a base, not a constraint. The local network at OWASP and BSides matters most for referrals to companies not on job boards.

Scope Creep into Red Team

AppSec engineers frequently perform application-level penetration testing as part of the role — Burp Suite, manual web app testing, OWASP methodology. This is expected and built into the plan. The drift to avoid is into full red team or network penetration testing, which is a distinct specialization requiring different skills (Active Directory, network exploitation, infrastructure pivoting). Red team is typically a destination role requiring demonstrated AppSec or pentest experience first — not a direct entry point from developer. Master the AppSec role first. The deeper offensive path opens more easily once you have AppSec experience on your resume.

Net assessment: The risks are real. None negate the market opportunity. The plan as designed — hands-on lab work over certifications, manual security skill over automated tooling familiarity, full-stack deployment capability — is structured to be resilient to the most likely downside scenarios.

Study Plan Phase Summary

This plan turns a working software engineer into an entry-level AppSec engineer over roughly eight to nine months. Rather than front-loading theory, it follows a deliberate arc: shore up the engineering base, get hands dirty with offense early, then build depth and professional tooling on top of that foundation.

It opens by refreshing the fundamentals a developer already half-knows — Python and FastAPI, HTTP, Docker, a working local lab (Phase 0) — and immediately follows with a first taste of the attacker mindset (Phase I), so security feels concrete from the start rather than abstract. It then fills the web and network gaps most developers skipped on the way to shipping features (Phase II).

The middle of the plan is the heart of the offensive skill set: going deep on the core vulnerability classes — injection, XSS, broken authentication and access control, SSRF, and the rest — by exploiting each one hands-on in deliberately vulnerable labs (Phase III), then layering on the professional toolchain (Burp Suite, plus SAST, DAST, dependency and secrets scanning) and earning the Burp Suite Certified Practitioner certification, the credential that proves real hands-on web-exploitation skill (Phase IV). A light thread of AI/LLM security — prompt injection, output handling, tool-calling risk — runs through these phases as well.

The back half pivots toward the work an AppSec engineer actually does day to day — cloud security basics (Phase V) and secure code review and threat modeling (Phase VI) — rounded out by CompTIA Security+ for breadth and to clear HR filters (Phase VII).

It closes with focused interview preparation (Phase VIII) and the job search itself (Phase IX). Applications begin in parallel from Month 4, though — well before the curriculum is finished — because a growing portfolio of writeups and a passed BSCP is what actually opens doors.

> **Phase 0 — Pre-Study: Foundations** (Wks 1–3). **Purpose:** quickly refresh and expand the engineering base before security content — Python/FastAPI, HTTP, Docker, a working lab. **Resources:** FastAPI docs, HTTP (Notes + MDN). **Achievement:** a complete FastAPI app built from scratch and a running local lab.
>
> **Phase I — AppSec Intro: A Dip into Offense** (Wks 4–5). **Purpose:** first taste of the attacker mindset — how web vulnerabilities are found and exploited. **Resources:** “Real-World Bug Hunting” (Ch 1–5), PortSwigger Web Security Academy, TryHackMe. **Achievement:** first labs and published lab writeups.
>
> **Phase II — Web & Network Foundations** (Mo 1.5–2.25). **Purpose:** deep HTTP, web, and authentication foundations plus Burp basics. **Resources:** “Alice and Bob Learn Application Security”, WAHH, Burp Suite docs, PortSwigger Authentication path. **Achievement:** Burp fluency, HTTP/auth/session mastery, and the PortSwigger Authentication, JWT, and OAuth labs cleared.
>
> **Phase III — Core Vulnerability Classes** (Mo 2.5–4.0). **Purpose:** master the core web vulnerability classes hands-on at practitioner depth. **Resources:** PortSwigger topic paths and their apprentice/practitioner labs, “Hacking APIs”, the OWASP Top 10. **Achievement:** practitioner labs across the OWASP classes and an OWASP Top 10 writeup series.
>
> **Phase IV — Security Testing & Tooling** (Mo 4.0–6.25). **Purpose:** tooling fluency — Burp mastery, SAST, DAST, SCA — and the BSCP certification. **Resources:** Burp docs, PortSwigger BSCP practice exams, Semgrep, Bandit, OWASP ZAP, Snyk. **Achievement:** passed the Burp Suite Certified Practitioner (BSCP) exam, completed a full Juice Shop security assessment.

Phase V — Cloud Security Fundamentals (Mo 6.25–6.75). **Purpose:** AWS security basics — shared responsibility, IAM, and common misconfigurations. **Resources:** flaws.cloud, flaws2.cloud, AWS IAM docs. **Achievement:** every level of the flaws.cloud and flaws2.cloud AWS security challenges completed, with writeups.

Phase VI — Secure Code Review & Threat Modeling (Mo 6.75–7.5). **Purpose:** the core AppSec craft — manual secure code review and threat modeling. **Resources:** OWASP Code Review Guide, OWASP ASVS, “Threat Modeling: Designing for Security”. **Achievement:** a documented code review and a threat model project.

Phase VII — CompTIA Security+ Certification (Mo 7.75–8.5). **Purpose:** earn the credential that clears HR and ATS filters; main application push. **Resources:** the “Get Certified Get Ahead: SY0-701” study guide and Professor Messer practice exams. **Achievement:** passed CompTIA Security+ (SY0-701).

Phase VIII — Interview Preparation & Interviewing (Mo 8.75–9.25). **Purpose:** convert skills and portfolio into offers — interview prep, story framing, application strategy. **Resources:** portfolio writeups and mock interviews. **Achievement:** a polished portfolio and interview narrative, plus active interviewing.

Phase IX — Expected First Days on the Job (Mo 9.0+). **Purpose:** start strong in the new sub-field and role and know your immediate differentiators. **Resources:** your accumulated notes and the five core reference books. **Achievement:** a written 30/60/90-day onboarding plan and early on-the-job wins.

0

Pre-Study: Foundations

Weeks 1–3 · Before Security Content Begins

Three parallel tracks before security content begins. The FastAPI track is the main one — 2–3 weeks to close out the official docs and build a complete project from scratch. The others are a few days each, run in parallel. All of this is active, hands-on work. By the end of Week 3, AppSec content starts immediately.

Learning Resources

- **FastAPI Official Documentation** — fastapi.tiangolo.com — Official docs, self-contained. Resume from Request Body section onward.
- **Existing HTTP Notes** — Personal notes (HTML intro + Odin HTTP). Cover request-response, methods, status codes, URL anatomy, headers, DNS, TCP/IP basics.
- **MDN Web Docs — Using HTTP Cookies** — developer.mozilla.org/en-US/docs/Web/HTTP/Cookies — Specifically for Secure, HttpOnly, and SameSite cookie attributes. The one HTTP topic not covered in existing notes.
- **Existing Docker Notes** — Personal notes and note cards covering containers, images, Dockerfiles, networking, volumes, Compose, Swarm, and Stack. More comprehensive than any online tutorial.
- **OWASP WebGoat** — github.com/WebGoat/WebGoat — Deliberately insecure web application covering OWASP Top 10 vulnerability classes with structured hands-on exploitation modules. Runs via Docker.

Pre-Study Schedule — Weeks 1–3

01. SETUP: Git & GitHub Hygiene · Day 1

- GitHub profile already exists with past projects. Before new portfolio work begins: confirm profile is public, bio is current, location is set to Philadelphia, PA.
- Review existing pinned repos — anything from the web dev years that still reflects well is worth keeping pinned. Anything that does not, unpin.
- Create a new repository for the FastAPI project. Commit as you build — clean messages, logical history. This repo is a portfolio artifact from day one and will be the most recent, actively developed project on the profile.
- Set up LinkedIn in parallel: headline and About section state the deliberate transition into application security as of June 2026. Certifications and portfolio links get added as they are earned and built.

02. REVIEW: HTTP & Cookies · Days 1–3

- **HTTP:** Review existing HTTP notes (PDF, .odt, and note cards). These cover the request-response model, methods (GET, POST, PUT, PATCH, DELETE), status codes (2xx, 3xx, 4xx, 5xx), URL anatomy, headers, DNS, and TCP/IP basics.
- **Cookies:** Cookies are not covered in the existing notes. Read MDN ‘Using HTTP cookies’ (developer.mozilla.org/en-US/docs/Web/HTTP/Cookies) specifically for the security attributes: Secure (only sent over HTTPS), HttpOnly (inaccessible to JavaScript), and SameSite (controls cross-site request

behavior). These three attributes appear throughout security review work.

• **SQL:** SQL is strong from prior study — joins, normalization, indexing. Do a fast review using existing SQL note cards.

03. READ + BUILD: FastAPI Documentation — Complete · Weeks 1–3

• The official docs cover Dependencies, Security, Middleware, CORS, and Request/Response internals — exactly the sections most relevant to AppSec work. Finishing the docs builds the complete mental model needed to audit production web codebases.

• **Starting point:** Start from the beginning. Re-read your notes from the covered sections (intro, path params, query params) first, then continue through the remaining tutorial sections. Slow down at Dependencies, Security, Middleware, and CORS — those four sections warrant the most detailed notes.

• **Build a personal FastAPI project as part of the reading:** Three years of adding to others' APIs is solid experience. Building one end-to-end — from blank file to deployed container — completes the picture. Design something small but complete: 5–6 endpoints, JWT authentication implemented via FastAPI's Security module, at least one dependency-injected auth check, a SQL database connection. The goal is to own a codebase fully from the ground up, which makes it the best candidate for security analysis work later in the plan.

04. LEARN + PRACTICE: Docker · Days 2–4

• Docker theory, Dockerfiles, networking, volumes, and Compose are all covered in existing Docker notes — which go further than any get-started tutorial, covering Swarm, Stack, and overlay networking. Review those notes rather than reading the online guide.

• **Practice:** Pull and run three containers: (1) nginx web server, (2) PostgreSQL, (3) OWASP WebGoat. Run, interact, stop, remove each one. Take notes on the commands.

• **Build:** Write a Dockerfile for the FastAPI project. Build it into an image, run it as a container. Closes the loop between the two parallel tracks.

Projects & Writings — Pre-Study

Project: FastAPI API — Built From Scratch

A complete FastAPI application: 5–6 endpoints, JWT auth, dependency-injected auth checks, SQL database. Committed to GitHub with clean history. First portfolio artifact.

Project: Dockerized FastAPI App

Dockerfile for the FastAPI project. Built into an image, run as a container.

Concepts Broached

FastAPI security internals

How FastAPI implements authentication and authorization via the Security module and Depends(). Where JWT validation happens. How middleware intercepts requests. How CORS is configured at the framework level. These are the layers AppSec engineers audit when reviewing application code for security issues.

Cookie security attributes

Secure, HttpOnly, and SameSite are not just configuration options — they are security controls. Their absence or misconfiguration is a finding in any security review.

Docker as a tool in AppSec

Containers provide isolation but also create attack surfaces: exposed ports, over-permissioned images, secrets in environment variables. Running WebGoat via Docker is the first hands-on encounter with containerized target environments, which appear throughout the plan.

I

AppSec Intro: A Dip into Offense

Weeks 4–5

AppSec has a mysticism problem: security culture sometimes makes it seem inaccessible without a decade of CTF grinding. That is not true, and this phase exists to demonstrate it directly — before any systematic study begins.

The goal is not to become a penetration tester in two weeks. It is to arrive at the Web & Network Foundations phase having touched real vulnerability classes with your hands, developed genuine questions about why they work the way they do, and established the foundational mental model: every input is a potential attack vector, and your job is to think about what happens when an attacker controls it.

Learning Resources

- **PortSwigger Web Security Academy** — portswigger.net/web-security — The most widely recommended AppSec learning resource among practitioners. Interactive labs, structured learning paths, every major vulnerability class covered. Built by the team that makes Burp Suite. Primary hands-on platform for the entire plan.
- **OWASP WebGoat** — github.com/WebGoat/WebGoat — A deliberately insecure application for learning. runs locally via Docker. Structured modules per vulnerability class.
- **TryHackMe** — tryhackme.com — Browser-based, beginner-friendly, guided rooms. The 'Web Fundamentals' path covers HTTP in Detail, Burp Suite: The Basics, and OWASP Top 10. The 'Jr Penetration Tester' path covers web vulnerabilities, Burp Suite, and penetration testing methodology.
- **"*Real-World Bug Hunting*"** — Peter Yaworski, No Starch Press, 2019 (nostarch.com) — A highly accessible introduction to web vulnerabilities. Real bug bounty reports, plain English, organized by vulnerability class.

AppSec Intro — Weeks 4–5

01. EXPLORE: PortSwigger Academy Orientation · Week 4

- Create a free PortSwigger account
- Skim the SQL injection material to get oriented, then complete the first 3–4 Apprentice & Practitioner level labs
- Skim the cross-site scripting material to get oriented, then complete the first 3–4 Apprentice-level labs (reflected and stored)

02. READ: "*Real-World Bug Hunting*" Ch. 1–5 · Weeks 4–5

- Chapter 1 covers bug bounty and HTTP basics; Chapters 2–5 are each a vulnerability class — open redirect, HTTP parameter pollution, CSRF, and HTML injection / content spoofing — illustrated with real disclosed bug bounty reports
- Read actively: when a report describes a finding in a Python or REST API context, pause and think about what the vulnerable pattern looks like in code — this builds the adversarial mental model

03. PROJECT: WebGoat — First Exploitation · Week 5

- Ensure local WebGoat still executes as expected in local Docker
- Complete these WebGoat lessons: SQL Injection (intro), Cross Site Scripting, Cross Site Scripting (stored), Authentication Bypasses, and Insecure Direct Object References (IDOR)
- For each module: exploit it, write your standard documentation (what, why, fix, code review pattern)

04. EXPLORE: TryHackMe Orientation · Week 5

- Complete three short Web Fundamentals rooms — HTTP in Detail, Burp Suite: The Basics, and OWASP Top 10
- These are short and designed for orientation, not mastery

Projects & Writings — AppSec Intro

Project: WebGoat Exploitation Log

For each WebGoat module completed, document: the vulnerability class, exploitation steps, root cause in the code, and the fix. First evidence of hands-on work in the portfolio.

Writing: What SQL Injection Taught Me About Trusting Input (900–1,200 words)

Plain-English explanation of what SQL injection is, why it works, and one thing that surprised you. Publish on LinkedIn when polished. Framing: ‘I have been building APIs for five years. Here is what I learned when I tried to break one.’

Concepts Broached

The OWASP Top 10

Introduced as a taxonomy. You have seen SQL injection and XSS in practice. The full Top 10 is a structured catalog of the most critical web vulnerability classes. Mastery of each class comes through dedicated study.

Injection as a class

Any time user input reaches an interpreter without proper sanitization, injection is possible. SQL injection is the canonical example. Command injection, LDAP injection, and template injection follow the same logic.

The attacker’s mindset

The mental shift from ‘does this work?’ to ‘what happens when I control this input?’ This is the foundational perspective change. You have started making it.

HTTP as the attack surface

Every web vulnerability lives in the HTTP layer — parameters, headers, cookies, request bodies. Understanding HTTP is the substrate for every vulnerability class in this plan.

How These Concepts Are Used as an AppSec Engineer

OWASP Top 10

This is the vocabulary of every AppSec interview and code review conversation. When a developer asks why their code is flagged, you explain which OWASP category it falls into and why it matters.

Injection mindset

When reviewing an API endpoint that accepts user input, your first question is always: does this input reach an interpreter? A database query? A shell command? A template renderer? This question catches more vulnerabilities than any automated tool.

HTTP literacy

Every Burp Suite session, every manual pentest, every DAST scan result is an HTTP transaction. Reading raw HTTP requests and responses fluently is required for the work.

II

Web & Network Foundations

Months 1.5–2.25

This phase builds the technical foundation for AppSec work. Every major vulnerability class lives inside HTTP, authentication protocols, browser security models, or API design patterns. The phase reframes these layers from an adversarial perspective — not just how they work, but why they fail. It closes with a focused final week on cryptography fundamentals and the specific networking knowledge that appears in AppSec interviews.

Learning Resources

- **"*Alice and Bob Learn Application Security*"** — Tanya Janca (SheHacksPurple), Wiley, 2020 (wiley.com) — 250 pages covering threat modeling, secure code review, SDLC security, and how to work with developers. Covers the defender-side, program-building angle that pure offensive resources miss.
- **"*The Web Application Hacker's Handbook*"** — Dafydd Stuttard and Marcus Pinto, Wiley, 2011 (wiley.com) — Foundational reference written by the creator of Burp Suite. Ch. 1–3 cover web application security, core defense mechanisms, and web application technologies. Ch. 6 covers authentication, Ch. 7 session management, Ch. 8 access controls. Some content is aging but these chapters remain the clearest treatment available.
- **PortSwigger Web Security Academy** — portswigger.net/web-security — Primary lab platform for this phase. Covers HTTP fundamentals, Burp Suite basics, authentication attacks, access control vulnerabilities, and SQL injection — all with interactive labs.
- **MDN Web Docs** — developer.mozilla.org — The authoritative reference for HTTP, cookies, CORS, CSP, and the browser security model. Covers each security header, its syntax, and its security purpose in depth.
- **OWASP SSRF Prevention Cheat Sheet** — cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html — Covers DNS rebinding attack mechanics, private IP range blocking patterns, and SSRF prevention guidance.
- **TryHackMe 'Pre-Security' path — Network Fundamentals module** — tryhackme.com/module/network-fundamentals — Primary resource for the networking week. Covers OSI model, TCP/IP, ports, and how packets travel in 3–4 interactive hours. Not Net+, not CCNA — precisely scoped for AppSec-relevant networking.
- **TryHackMe 'Pre-Security' path — Putting It All Together room** — tryhackme.com/room/puttingitalltogether — Covers how all the components of a web request fit together: DNS, HTTP, load balancers, CDNs, databases, and how each layer relates to security decisions.
- **OWASP Cryptographic Storage Cheat Sheet** — cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html — Primary reference for the cryptography week. Covers which algorithms are acceptable and which are broken, key management, and storage patterns.
- **OWASP Transport Layer Security Cheat Sheet** — cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html — TLS configuration, certificate management, and common misconfigurations. Companion to the crypto week.

- **hashcat** — hashcat.net/wiki — Password cracking tool used in the cryptography week to crack an MD5 hash and make weak hashing concrete. The wiki includes a beginner page with exact example commands for MD5 wordlist attacks.
- **bcrypt (Python library)** — pypi.org/project/bcrypt — Python library for bcrypt password hashing. Used in the cryptography week to implement correct password hashing in Python. Simple API: hash a password, verify a password.

Web & Network Foundations — Months 1.5–2.25

01. READ + LABS: HTTP Deep Dive · Week 6

- Read *WAHH* Ch. 1–3 (web application security, core defense mechanisms, web application technologies) as foundational context
- Install Burp Suite Community Edition, then work through the official documentation — Proxy, Repeater, and Intruder — the primary tools you will use throughout this phase
- Read CORS misconfigurations, CSP, TLS, HTTP methods, and status codes for their security implications
- **Full notes + note cards:** CORS misconfigurations, CSP, TLS, HTTP methods, and status codes with security implications

02. READ + LABS: Authentication & Session Management · Weeks 6–7

- Read *WAHH* Ch. 6 (Attacking Authentication) and Ch. 7 (Attacking Session Management)
- Complete the PortSwigger Authentication learning path — all apprentice and practitioner labs
- Complete the JWT learning path, then its labs — unverified signature, flawed signature verification, weak signing key, jwk / jku injection, and *algorithm confusion*; then the OAuth learning path and its labs — implicit-flow bypass, forced profile linking, and *account hijacking via redirect_uri*
- **Full notes + note cards:** Password enumeration, session fixation, JWT algorithm confusion, OAuth redirect URI manipulation

03. PROJECT: Authentication Audit Checklist · Week 7

- Write a 1–2 page authentication audit checklist from scratch — no templates
- Every item should come from a concept in your notes
- For each item: what you check, how you check it (manually and with what tool), and what a finding looks like
- Run the checklist against the FastAPI app you built in Phase 0 — it implements JWT authentication via FastAPI's Security module, so it is the ideal first audit target. Record any findings.
- Push to GitHub as Markdown. First real security artifact in the portfolio.

04. READ + LABS: Access Control & APIs · Week 7

- Read *WAHH* Ch. 8 (Attacking Access Controls)
- Read the PortSwigger Access control and API testing learning paths, then work the apprentice-level Access control labs and API testing labs to build the foundational mental model — the practitioner labs follow in Phase III

- **Full notes + note cards:** IDOR, horizontal/vertical privilege escalation, mass assignment, excessive data exposure

05. READ + LABS: Input Handling Fundamentals · Weeks 7–8

- Read the PortSwigger SQL injection learning path, then work the apprentice-level labs to internalize the input-validation root cause — the practitioner labs follow in Phase III
- **Full notes + note cards:** SQL injection root cause, parameterized queries, injection taxonomy

06. READ: "*Alice and Bob Learn Application Security*" · Week 8

- Read "*Alice and Bob Learn Application Security*" Ch. 1–5 and Ch. 7: security fundamentals, security requirements, secure design, and common pitfalls (Ch. 1–5), and what an AppSec program does day-to-day (Ch. 7)
- Read during this week — this is the defender and program-building perspective that PortSwigger and *WAHH* do not cover
- **Key insight:** AppSec is not just finding vulnerabilities — it is building processes so developers stop introducing them
- **Full notes + note cards:** SDLC security touchpoints, developer security hygiene checklist, what an AppSec program actually does day-to-day

07. STUDY: Cryptography Fundamentals · Week 9

- Review existing sys design notes on auth and basic security before any new reading. The goal this week is the AppSec angle: how cryptographic primitives fail, what broken implementations look like in code, and how to flag them in a security review.
- Read the OWASP Cryptographic Storage Cheat Sheet and the OWASP Transport Layer Security Cheat Sheet. Take notes.
- **Hands-on:** Crack an MD5 hash with hashcat. Implement bcrypt password hashing in Python. These make the concepts concrete.
- **Full notes + note cards:** One card per algorithm type covered in the cheat sheets. Front: name and use case. Back: when to use it, what breaks it, what a broken implementation looks like in code.

08. STUDY: Networking Fundamentals for AppSec · Week 9

- Complete the TryHackMe 'Pre-Security' path — Network Fundamentals module. Take notes.
- Review existing notes for the URL-to-response flow.
- Read the OWASP SSRF Prevention Cheat Sheet — specifically the DNS rebinding section.
- Complete the TryHackMe 'Pre-Security' path — Putting It All Together room.
- **Full notes + note cards:** OSI layers 3/4/7 with one AppSec example each; TCP vs UDP with one security implication each; the URL request flow; common ports; private IP ranges; what a firewall, load balancer, and reverse proxy each do.

09. WRITE: Authentication Writeup + LinkedIn Post · Week 9

- Publish 800–1,200 word LinkedIn article: 'Five JWT vulnerabilities I found while reviewing authentication code'
- Use examples from the PortSwigger labs. Frame it as a developer who learned to break what they built

Concepts Mastered

HTTP security model

Every HTTP header that has security implications, what it does, and what happens when it is misconfigured. The substrate of every web vulnerability.

Authentication and session security

How authentication protocols work and how they fail. JWT algorithm confusion alone appears in more real breaches than most developers realize.

Access control

IDOR, privilege escalation, horizontal vs. vertical access control flaws. The most common vulnerability class in production applications in 2026.

API security

REST API-specific vulnerability patterns — broken object-level authorization (IDOR), mass assignment, and excessive data exposure — and why APIs widen the attack surface beyond traditional server-rendered applications.

Input validation

The root cause of injection vulnerabilities. Every injection class shares the same root cause: unsanitized user input reaching an interpreter.

Cryptography fundamentals

Hashing vs. encryption vs. encoding. Which algorithms are broken (MD5, SHA-1 for passwords, DES, RC4) and which are correct (bcrypt, AES-GCM, RSA). How to identify weak crypto in a code review without being a cryptographer.

Networking fundamentals for AppSec

OSI layers 3 (Network/IP), 4 (Transport/TCP-UDP), and 7 (Application/HTTP) and why AppSec lives at layer 7. TCP vs UDP and one security implication of each. The seven steps of a URL request from DNS query to HTTP response. Common ports and the services on them. Private IP ranges (10.x, 172.16.x, 192.168.x), localhost (127.0.0.1), and the AWS metadata endpoint (169.254.169.254). Standard web app network architecture: internet → firewall → load balancer → app server → database. What a firewall, load balancer, and reverse proxy each do and what security question to ask about each.

Networking as an attack surface

DNS rebinding exploits the gap between DNS validation and TCP connection. SSRF against private IP ranges and 169.254.169.254 is the most common high-severity cloud vulnerability. Understanding where TLS terminates in a web architecture tells you whether internal traffic is encrypted. Open unexpected ports in a deployment configuration are a finding.

Networking as an attack surface

DNS resolution flow and why DNS rebinding bypasses SSRF protections. The AWS metadata endpoint (169.254.169.254) and why SSRF against it is a critical-severity finding. Why HTTP statelessness makes cookies necessary and therefore an attack surface. These are not networking engineering concepts — they are the specific networking behaviors that AppSec engineers encounter in real work.

AppSec program thinking (Janca)

AppSec is not just finding bugs — it is building the processes, training, and SDLC integration that stop bugs from being introduced. The developer’s perspective on the defensive side of the role.

How These Concepts Are Used as an AppSec Engineer

HTTP headers in code review

When reviewing a PR that sets cookies or configures CORS, you check HttpOnly, Secure, SameSite, and Access-Control-Allow-Origin as a reflex. Most developers do not know these flags exist.

JWT review

JWT configuration is the first thing you check: algorithm whitelisted? Signature verified? Claims validated? Three questions catch the most common JWT vulnerabilities.

IDOR in API review

When reviewing an API endpoint that returns data based on a user-supplied ID, you ask: is the user's authorization checked against the requested resource, or just their authentication? This question catches IDOR every time.

Cryptography in code review

When you see a password hash, you check whether it is bcrypt/Argon2 (correct) or MD5/SHA-1 (flag immediately). When you see encrypted data, you check for AES-GCM (correct) vs AES-ECB (flag). When you see a secret, you check whether it is hardcoded or pulled from a secret manager. These are reflex checks that take seconds once the vocabulary is owned.

Networking in architecture review and interviews

When asked 'where does TLS terminate?' in an interview or architecture review, you can answer and explain the security implication. When reviewing a deployment configuration, you check for unexpected open ports. When a threat model shows a service making outbound HTTP calls, you ask whether SSRF mitigations are in place and whether private IP ranges are blocked. When an interviewer describes traffic from an EC2 instance to 169.254.169.254 and asks if it is concerning, the answer is yes — SSRF against the metadata endpoint.

III

Core Vulnerability Classes

Months 2.5–4.0

Phase III is the technical core of the plan. The OWASP Top 10 is the field's shared vocabulary for the most critical web application security risks. Every AppSec interview assumes you know these. Every code review uses them as a checklist. Every penetration test methodology is organized around them. This phase goes deep on each class — not survey knowledge, but genuine understanding of root cause, exploitation mechanics, and remediation. Complete every Apprentice and Practitioner lab in each learning path below. Expert labs are stretch goals.

Learning Resources

- **PortSwigger Web Security Academy** — portswigger.net/web-security — Primary lab platform for this phase. Interactive labs covering every major vulnerability class: injection, XSS, CSRF, SSRF, XXE, access control, business logic, deserialization, file upload, path traversal, and API testing.
- **OWASP Top 10 (Web)** — owasp.org/www-project-top-ten — The ten most critical web application vulnerabilities. Each entry includes a description, example attack scenarios, and prevention guidance.
- **OWASP API Security Top 10** — owasp.org/www-project-api-security — The API-specific companion to the web Top 10. Covers BOLA (Broken Object Level Authorization), BFLA (Broken Function Level Authorization), broken authentication, excessive data exposure, and SSRF as they manifest in REST APIs.
- **OWASP Testing Guide v4.2** — owasp.org/www-project-web-security-testing-guide — The methodology reference. For each vulnerability class, it describes how to test systematically.
- **"*Alice and Bob Learn Application Security*"** — Tanya Janca (SheHacksPurple), Wiley, 2020 (wiley.com) — Ch. 6–8 cover common pitfalls, securing modern applications, and building an AppSec program.
- **"*Hacking APIs*"** — Corey Ball, No Starch Press, 2022 (nostarch.com) — REST API security testing. Ch. 1–3 and Ch. 6–9 cover API reconnaissance, endpoint discovery, and authentication testing.
- **OWASP Top 10 for LLM Applications** — genai.owasp.org/llm-top-10 — the standard risk list for LLM-backed apps: prompt injection, improper output handling, excessive agency, and sensitive information disclosure.

Core Vulnerability Classes — Months 2.5–4.0

01. READ + LABS: Injection Classes: Weeks 10–11

- **Buy Burp Suite Professional now** (~$499/yr) — the out-of-band labs in this phase need Burp Collaborator, which is Pro-only. Buying at the start of Phase III lets you do every lab here in place instead of skipping the Collaborator-based ones, and you use Pro continuously from here through the BSCP exam. See Burp Suite Professional.
- Re-read your Phase II SQL injection notes (the learning path stays linked if you need it), then complete the practitioner labs — the UNION-based and blind (boolean- and time-based) techniques, plus the out-of-band labs (Burp Collaborator — you now have Pro)

- Work through the PortSwigger OS command injection and Server-side template injection learning paths, then their labs — the OS command injection labs (the simple case plus blind time-delay and output-redirection, and out-of-band exfiltration via Collaborator) and the basic SSTI labs only (enough to recognize SSTI in review; skip the engine-specific gadget exploitation)
- **For each lab:** exploit it, write your standard documentation (what, why, fix, code review pattern)

02. PROJECT: OWASP Top 10 Writeup Series: Throughout Weeks 10–14

- One 800–1,200 word writeup per OWASP Top 10 category, published progressively throughout this phase
- Not academic summaries — structured as: what it is, real-world example, how you identified it in a lab, how to fix it, what to look for in code review
- These ten writeups are the backbone of your AppSec portfolio

03. READ + LABS: Cross-Site Scripting: Week 11

- Complete the PortSwigger Cross-site scripting learning path, then the labs covering reflected, stored, and DOM XSS, the DOM-sink variants, and the exploitation labs (stealing cookies, XSS-to-CSRF); skip the long tail of near-identical context-encoding and filter-bypass permutations
- **XSS in React:** React's JSX escaping prevents many reflected XSS vectors by default. The key exception is dangerouslySetInnerHTML, which bypasses JSX escaping entirely and is a standard code review finding.
- **Key:** XSS to account takeover chains — cookie theft, CSRF token theft, keylogging

04. READ + LABS: CSRF, SSRF, XXE: Weeks 11–12

- Complete the PortSwigger CSRF, SSRF, and XXE learning paths, then their labs — the CSRF labs in full (each is a distinct broken-defense pattern), the apprentice and practitioner SSRF labs (skip the two expert labs — Shellshock and the whitelist-filter bypass), and the core XXE labs (file retrieval, SSRF, and one blind example — all apprentice and practitioner)
- **SSRF:** a vulnerability where an attacker tricks a server into making HTTP requests to unintended destinations — including internal services and cloud metadata endpoints. API endpoints that fetch user-supplied URLs are the primary SSRF surface. The exploitation mechanics are covered in this phase.
- Cloud metadata service attacks (169.254.169.254) are the canonical SSRF exploitation target

05. READ + LABS: Access Control & Business Logic: Week 12

- Review your Phase II Access control notes, then complete the remaining practitioner labs — the URL-, method-, and Referer-based bypasses and multi-step gaps
- Work through the PortSwigger Business logic vulnerabilities learning path, then its apprentice and practitioner labs — each is a distinct logic flaw, and this is the class your developer background most directly advantages, so cover it well
- Business logic is the vulnerability class automated tools miss entirely — developer background is the structural advantage here

06. READ + LABS: Deserialization, File Upload, Path Traversal: Week 13

- Complete the PortSwigger Insecure deserialization, File upload, and Path traversal learning paths, then their labs — the deserialization labs concept set only (modifying serialized data, using application functionality, one gadget-chain example; skip the extra language-specific gadget chains), the apprentice and practitioner file upload labs in full, including the expert web-shell-via-race-condition lab (Burp Pro), and

the path traversal labs in full (each is a distinct filter bypass)

- Less common than injection and XSS but high-severity in enterprise codebases when present

07. READ + LABS: API Security Deep Dive: Weeks 13–14

- Read the OWASP API Security Top 10 in full
- Key additions over the web Top 10: BOLA (Broken Object Level Authorization = IDOR for APIs), BFLA (Broken Function Level Authorization), Mass Assignment, Excessive Data Exposure
- Read "*Hacking APIs*" Ch. 1–3 (how web applications and APIs work, and the common API vulnerability classes) and Ch. 6–9 (API discovery and reconnaissance, endpoint analysis, authentication attacks, and fuzzing)
- Review your Phase II API testing notes, then complete the remaining practitioner labs — BOLA, mass assignment, and excessive data exposure
- Work through it treating your own past API work as the mental target: would the APIs you built have been vulnerable?

08. READ + LABS: Web LLM Attacks (PortSwigger): Weeks 15–16

- Securing LLM-backed features is a fast-growing AppSec differentiator, and it builds directly on your API background
- Web LLM attacks learning path, then the labs — Exploiting LLM APIs with excessive agency, Exploiting vulnerabilities in LLM APIs, Indirect prompt injection, and the AI-powered scanner labs

08. READ: OWASP Top 10 for LLM Applications: Week 16

- Now that you have exploited the web-facing LLM classes hands-on, read the OWASP Top 10 for LLM Applications straight through as the framework that names and organizes them
- Map what you did to the list, in order: prompt injection (LLM01), sensitive information disclosure (LLM02), improper output handling (LLM05), and excessive agency (LLM06) are the classes you attacked; the rest (supply chain, data and model poisoning, system prompt leakage, vector and embedding weaknesses, misinformation, and unbounded consumption) are the broader categories to recognize

Concepts Mastered

Injection — complete taxonomy

SQL, command, template, LDAP, NoSQL. Root cause: unsanitized user input reaching an interpreter. Fix: parameterized queries. Code review: any string concatenation that includes user input reaching an interpreter.

XSS — all three types

Reflected, stored, DOM. Root cause: user input rendered as HTML without encoding. **Fix:** context-appropriate output encoding. Code review: any innerHTML assignment, dangerouslySetInnerHTML, server-side template rendering of user data.

CSRF and SSRF

CSRF exploits browser trust. SSRF exploits server trust to reach internal resources. **Fix:** SameSite cookies for CSRF; URL allowlists for SSRF.

Access control flaws

The most common finding in production AppSec work. Fix: verify the authenticated user is authorized for the specific resource on every request.

Business logic vulnerabilities

Not detectable by automated tools. Requires understanding intended behavior. The developer background is the advantage.

How These Concepts Are Used as an AppSec Engineer

Injection in code review

Every database query, every shell command, every template render gets the same question: does user input reach this without sanitization? One question catches the whole class.

XSS in React code review

React escapes JSX output by default, which prevents many reflected XSS vectors. The three things to check in a React code review: dangerouslySetInnerHTML usage (bypasses JSX escaping entirely), third-party component XSS vectors, and any server-rendered template output that feeds into the React tree.

SSRF in API review

Any endpoint accepting a URL parameter is flagged immediately. Does it fetch that URL server-side? Is the response returned to the user? Is there a URL allowlist? Three questions.

Business logic in threat modeling

This is the vulnerability class that threat modeling catches before it ships. STRIDE analysis of a data flow diagram surfaces the business logic assumptions that could be violated.

Securing LLM-backed features

An increasingly common AppSec task. The LLM-specific classes — prompt injection, insecure output handling, excessive agency, and sensitive information disclosure — sit on top of the classic vulnerabilities you already review, and map onto skills you have: injection is injection, output handling is untrusted-input handling, excessive agency is least privilege.

IV Security Testing & Tooling

Months 4.0–6.25

Phase IV moves from knowing vulnerability classes to operating the professional toolchain used to find them systematically. Burp Suite is the primary tool. SAST (static analysis of source code, via Semgrep) and DAST (dynamic testing of the running app) are the automation layer. The goal is to reach a level of tool fluency where tooling becomes leverage, not a learning distraction — you use Burp Suite to find vulnerabilities faster, not to compensate for not understanding them.

Learning Resources

- **Burp Suite Documentation** — portswigger.net/burp/documentation — Official documentation for all Burp Suite tools: Proxy, Repeater, Intruder, Sequencer, and Scanner.
- **Semgrep Documentation** — semgrep.dev/docs — Primary for general SAST. The leading open-source SAST tool, most commonly referenced in AppSec job postings. Sufficient for the entire plan.
- **Bandit Documentation** — bandit.readthedocs.io — Python-specific SAST tool. Detects insecure subprocess calls, eval() usage, weak SSL contexts, hardcoded passwords, insecure pickle and yaml usage.
- **OWASP ZAP Documentation** — zaproxy.org/docs — the free, open-source OWASP DAST tool: an intercepting proxy plus an automated scanner that crawls a running app and runs active and passive checks for common web vulnerabilities. A free alternative to Burp Scanner, with a baseline scan that wires easily into CI.
- **Snyk Documentation** — docs.snyk.io — Software composition analysis (SCA) — finding vulnerabilities in third-party dependencies. Available with a free tier.
- **PortSwigger Web Security Academy** — portswigger.net/web-security — Primary resource for Burp Suite mastery and BSCP preparation. The Burp Suite learning path and BSCP practice exams live here.
- **OWASP WebGoat** — github.com/WebGoat/WebGoat — Deliberately insecure application. Used as a SAST scan target — run Semgrep and Bandit against its Python source to practice identifying real vulnerability patterns.
- **OWASP Juice Shop** — owasp.org/www-project-juice-shop — Deliberately insecure modern web application. The primary target for this phase's full security assessment: Burp Suite manual testing, ZAP DAST scan, Semgrep SAST, Snyk SCA. Runs via Docker or Node.
- **TruffleHog** — github.com/trufflesecurity/trufflehog — Secrets scanner that searches full git history for leaked credentials, including credentials committed and later deleted.
- **Gitleaks** — github.com/gitleaks/gitleaks — Lightweight secrets scanner. Runs as a pre-commit hook and CI gate. Scans diffs rather than full history, making it fast enough for every PR.

Security Testing & Tooling — Months 4.0–6.25

01. LEARN + LABS: Burp Suite Mastery: Weeks 17–18

- Proxy, Repeater, and Intruder are already yours from Phase II — review those notes; no need to re-read the docs
- The two Community-edition tools you haven't used: Decoder (encode/decode/hash inline) and Comparer (visual diff of two messages)
- **Decoder:** SQL injection with filter bypass via XML encoding (Essential skills) — encode the payload as XML/HTML entities to bypass the WAF
- **Comparer:** Username enumeration via different responses (Authentication) — spot the one login response that differs
- **Both:** Blind SQL injection with conditional responses (SQL injection) — read the admin password one true/false answer at a time
- **By end of Week 18:** fluent with Decoder and Comparer, and comfortable with Proxy match-and-replace and Repeater tab groups

02. PROJECT: Burp Suite Certified Practitioner Prep: Weeks 17–19

- The BSCP exam is highly respected in the AppSec community — practical, hands-on, and not multiple-choice
- **Burp Suite Professional required:** the exam, its practice exams, the targeted Scanner, and Burp Collaborator (for out-of-band work) all need Burp Suite Professional (~$499/yr) — Community Edition is not sufficient. If you followed the plan you bought Pro at the start of Phase III, where the first Collaborator labs appear; it is required continuously from there through this exam. The exam itself is ~$99 per attempt.
- The practice exam format (identify two vulnerabilities per app, chain them into account takeover) is the closest simulation of real AppSec work available
- Pass the BSCP practice exam in under 2 hours — and do it more than once — before booking the real exam. It mirrors the real format exactly, so consistently passing it under time is the strongest signal you are ready
- A passed BSCP on the resume is a much stronger AppSec signal than Sec+ alone

03. DOCUMENT: Save Exam-Ready Proof-of-Concept Exploits: Throughout Weeks 17–24

- PortSwigger publishes full PoC / solution code for every lab. As you clear each practitioner lab this phase, save its working exploit into one personal, searchable notes file — the BSCP is open to your own notes (see PortSwigger's prep guidance).
- Digital notes are ideal — the exam is fully open-book (your own notes, the web, third-party tools, and Burp extensions are all allowed) and you sit it in Chrome alongside Burp, so keep everything in one searchable file (Markdown or plain text) or a notes app rather than on paper. Make payloads copy-paste-ready, and pre-load your payload lists and useful extensions into Burp before you start.
- Pre-adapt the high-value PoCs for exam conditions: turn XSS print()/alert() payloads into cookie-stealers, parameterize your SQLi and SSRF payloads, and keep Collaborator / OOB templates ready to paste.
- Organize them by vulnerability class so you can pull the right template in seconds under exam time pressure

04. LEARN + LABS: Semgrep & Bandit — SAST Tooling: Weeks 18–19

- Install the Semgrep CLI. Run the default Python ruleset against a sample Python application (OWASP Juice Shop or WebGoat codebase)

- Triage the results: distinguish true positives from false positives
- Write one custom Semgrep rule targeting a Python injection pattern — for example, an endpoint passing request parameters directly to a database query
- Integrate Semgrep into a GitHub Actions workflow (sample repo is fine)
- Bandit: install bandit with pip. Run against a sample Python project and against some code in old projects. Look for: B101 (assert usage), B102 (exec usage), B105/B106/B107 (hardcoded password), B201 (Flask debug mode), B301 (pickle), B501–B509 (SSL/TLS issues). The finding IDs become vocabulary you use in code review conversations
- Run both tools against the same Python codebase and compare results. Bandit catches Python-specific antipatterns that Semgrep misses without custom rules. Semgrep catches logic-level patterns that Bandit's fixed ruleset cannot express.

05. LEARN: Secrets Scanning — TruffleHog & Gitleaks: Week 19

- Hardcoded secrets (API keys, passwords, tokens committed to repos) are one of the most common real-world breach vectors. The Uber 2016, LastPass 2022, and Toyota 2023 breaches all involved exposed credentials
- TruffleHog: install and run against a sample repo. Uses entropy analysis to catch secrets that regex misses. Can scan full git history, not just current code
- Gitleaks: install and run as a CI pre-commit hook. Lightweight and fast, well-suited for blocking commits before they land
- Integrate both into a GitHub Actions workflow: Gitleaks on every pull request (scanning only the changed lines), TruffleHog for scheduled full-history scans
- **In code review:** you are now looking for hardcoded strings that look like keys, passwords, or tokens — in source code, config files, Dockerfiles, and environment variable defaults
- **Full notes + note cards:** When to use TruffleHog vs Gitleaks, what a false positive looks like, what to do when a live secret is found (rotation procedure, not just deletion)

06. LEARN: OWASP ZAP — DAST Basics: Week 19

- Run OWASP ZAP against DVWA or WebGoat running locally
- Understand the difference between DAST (testing a running application) and SAST (testing source code)
- **Triage ZAP results:** true positive, false positive, needs investigation

07. LABS: Out-of-Band (OOB) Exploitation with Burp Collaborator: Weeks 19–20

- **Requires Burp Suite Professional** — Burp Collaborator is Pro-only, and nearly every blind / out-of-band variant on the BSCP routes through it. You first used Collaborator in Phase III (Pro from the start); this step is a focused OOB drill to lock in exam fluency — re-confirm each one fast.
- **Blind SQL injection:** re-read your notes on blind SQL injection (the learning path stays linked), then the out-of-band labs — *Blind SQL injection with out-of-band interaction* and *Blind SQL injection with out-of-band data exfiltration* (practitioner)
- **Blind XXE:** re-read your notes on blind XXE (the learning path stays linked), then the out-of-band labs — *Blind XXE with out-of-band interaction* and *Exploiting blind XXE to exfiltrate data using a malicious external DTD*

- **Blind OS command injection:** from the OS command injection learning path, the out-of-band labs — *Blind OS command injection with out-of-band interaction* and *... with out-of-band data exfiltration*
- Carry the same Collaborator technique into blind SSTI detection and any asynchronous sink — fire a DNS/HTTP callback to confirm the bug, then exfiltrate data through it
- **For each lab:** exploit it, then save the Collaborator payload and working exploit into your exam notes

08. READ + LABS: Remaining BSCP Topics: Weeks 19–21

- **Topics the exam can draw from that earlier phases skipped — close them before sitting the exam.** For each: read the learning path, then do the apprentice and practitioner labs.
- HTTP request smuggling learning path, then the labs — *basic CL.TE* and *basic TE.CL*, *obfuscating the TE header*, and *Exploiting HTTP request smuggling to capture other users' requests* (a BSCP favorite)
- HTTP Host header attacks learning path, then the labs — *basic password reset poisoning*, *Host header authentication bypass*, *web cache poisoning via ambiguous requests*, and *routing-based SSRF*
- Web cache poisoning learning path, then the labs — *unkeyed header*, *unkeyed cookie*, *multiple headers*, and *targeted poisoning using an unknown header*

09. LEARN + LABS: Targeted Scanning & Content Discovery: Weeks 20–21

- **Requires Burp Suite Professional** (the Scanner and Engagement tools are Pro-only). The exam is time-boxed, so you scan smart, not broad — this is an explicit BSCP skill.
- Read using Burp Scanner during manual testing, then the labs — *Discovering vulnerabilities quickly with targeted scanning* and *Scanning non-standard data structures*. Right-click a single request → *Do active scan* to audit just that request in seconds instead of crawling the whole site.
- **Content discovery:** use Burp's Engagement tools → *Discover content* to enumerate hidden directories, files, and parameters (for example a hidden /admin) — a recurring first move on exam apps. Practice it on labs you have already solved.
- **Build the habit:** hand every new request to the Scanner while you test manually, and run content discovery before assuming an app's surface is fully mapped

10. DRILL: PortSwigger's Mandatory Reinforcement Labs: Weeks 22–23

- Beyond one practitioner lab per topic, PortSwigger requires 8 specific skill-reinforcement labs before the exam. Make sure each is solid — several are already covered in earlier steps; this is the consolidated checklist.
- **XSS:** *Exploiting cross-site scripting to steal cookies* — from Cross-site scripting, the labs; exfiltrate a real session cookie (not just an alert()), which is what the exam rewards (reinforces step 03)
- **Authentication:** *Brute-forcing a stay-logged-in cookie* — from Authentication, the labs; decode the cookie in Decoder, then brute-force it with Intruder
- **SSRF:** *SSRF with blacklist-based input filter* — from SSRF, the labs; the filter-bypass variant skipped in Phase III
- Already done elsewhere — re-confirm you can solve each cold: *Forced OAuth profile linking* (Phase II), *Exploiting HTTP request smuggling to capture other users' requests* (step 08), *Blind SQL injection with out-of-band data exfiltration* (step 07), *SQL injection with filter bypass via XML encoding* (step 01), and *Discovering vulnerabilities quickly with targeted scanning* (step 09)
- Save the working PoC for each into your exam notes — you may refer to your own notes during the exam

11. **DRILL: Fluency — Re-Solve Cold:** Week 23
    - For each major class, take one practitioner lab you have already solved and re-solve it cold — no notes, no solution — in 15 minutes or less.
    - Work through roughly 20 labs this way (about one per class). If a lab runs past 15 minutes or you reach for your notes, that class is not yet exam-fluent — drill it again.
    - This is the gap between having seen a vulnerability and being able to exploit it under time pressure with no hints — which is exactly what the exam tests
12. **DRILL: Mystery Labs — Cold Recognition:** Weeks 23–24
    - The exam gives you no vulnerability hints — you must recognize the class from behavior alone. The mystery lab challenge spins up a random lab with its title and description hidden, the closest practice for that.
    - Solve at least 5 mystery labs cold — no hints, working only from recon — before booking the exam. Time each one; the goal is fast, confident classification.
    - If a class keeps stumping you, go back to that topic's labs and notes, then return to the mystery challenge
13. **DRILL: Exam Stamina — Six Labs in One Block:** Week 24
    - The real exam is a single continuous 4-hour session — two applications, three stages each, six flags, no partial credit. Rehearse that endurance before you book it.
    - Take six practitioner labs you have not memorized and solve them back-to-back in one timed 4-hour block — own notes only, Burp Pro, Collaborator live, no long breaks.
    - Track time per lab and practice triage: if one runs past budget, bank partial progress, move on, and circle back — the time management is as much the test as the exploitation
14. **EXAM: Burp Suite Certified Practitioner:** Weeks 24–25
    - Take the BSCP exam. The exam consists of two applications, each requiring two vulnerabilities to be identified and chained into an account takeover. 4 hours.
    - If unsuccessful on the first attempt, review the topics where the attempt failed and retake. There is no cap on attempts — you can retake as many times as needed (each attempt costs $99).
15. **APPLY: Begin Job Applications:** Month 4 onward
    - With Phases I–III complete, two or more published writeups, and visible PortSwigger progress, begin applying now — do not wait for curriculum completion
    - Target AI-native startups, security-conscious tech companies, and local Philadelphia legal-AI and healthtech firms first — they hire locally and value a full-stack background
    - Job applications run in parallel with the remaining phases and ramp into a main push around Month 8.5, once Security+ is on your resume
    - See "Application Strategy" in Phase VIII for the full targeting, profile, and framing playbook
16. **LEARN: Snyk — Dependency Scanning:** Week 24
    - Install and authenticate the Snyk CLI, then run *snyk test* in a sample Python project with known-vulnerable dependencies (for example an outdated requirements.txt) to produce a software-composition report listing each flagged package, its CVE, and the safe upgrade

- Understand what SCA (software composition analysis) is and what it does not catch
- Log4Shell was a dependency vulnerability, not an application vulnerability — SCA catches these
- Wire *snyk test* into CI alongside Semgrep and Gitleaks so newly disclosed dependency CVEs fail the build — the remediation is almost always a version bump

17. PROJECT: Full Security Assessment of Juice Shop: Week 25

- Conduct a full security assessment of OWASP Juice Shop using: Burp Suite (manual), ZAP (DAST), Semgrep + Bandit (SAST), Snyk (dependencies), TruffleHog/Gitleaks (secrets)
- Work through it systematically using the official companion guide Pwning OWASP Juice Shop, which groups every challenge by vulnerability class — injection, broken authentication, XSS, broken access control, and more — so you can map each finding to an OWASP category; its appendix has full step-by-step solutions if you get stuck
- Reuse the systematic methodology and the writeup and report format you built in your Phase III vulnerability-class writeups — this project is where the full workflow comes together: recon, tooling, manual exploitation, false-positive triage, and professional reporting
- Produce a written vulnerability report: executive summary, findings table (vuln, severity, CVSS, evidence, recommendation), detailed finding writeups
- Export the finished report as a polished, shareable standalone PDF — the document you attach to applications and hand to interviewers
- This is the most important portfolio artifact in the plan — it demonstrates the complete AppSec workflow from tool setup to professional reporting

18. PROJECT: Secure a Small LLM-Backed API: Week 25

- Add one small LLM-backed endpoint to your Phase 0 FastAPI app — a /summarize or support-reply route that calls a hosted model API (for example the Anthropic or OpenAI API). This is the flagship AI portfolio piece; it builds directly on your API background
- Run the same three attack classes you practised in your Phase III Web LLM Attacks step against your own endpoint: prompt injection (LLM01); improper output handling (LLM05) — show the XSS or SQL injection that trusting model output enables, then fix it with encoding and parameterization; and excessive agency (LLM06) — least privilege plus an approval step for any tool the model can call
- Write it up in your standard report format and push it to GitHub; label each finding with its OWASP LLM Top 10 ID (LLM01, LLM05, and so on). One focused ‘securing an LLM API’ writeup is a strong, current portfolio piece
- Scope: this is securing an LLM *application*, not machine learning — stay on the AppSec slice

Concepts Mastered

Burp Suite as an extension of manual thinking

Burp Suite intercepts and replays HTTP traffic. It does not find vulnerabilities — it makes manual exploitation efficient. Understanding of what to look for drives the tool, not the reverse.

SAST vs. DAST vs. SCA vs. Secrets Scanning

Four distinct scanning approaches that catch different vulnerability classes. SAST reads code, misses runtime behavior. DAST tests a running app, misses code paths not exercised. SCA finds known CVEs in dependencies. Secrets scanning finds exposed credentials in code and history. A real AppSec program uses all four — plus Python-specific tools like Bandit for Python shops.

False positive triage

Automated tools have noise. Distinguishing real findings from false positives without dismissing everything is where security judgment matters more than tool knowledge.

Professional vulnerability reporting

Severity rating (CVSS), evidence documentation, clear remediation guidance. The output of AppSec work is a finding that a developer can act on — not just a list of things that are broken.

Secrets management

Hardcoded credentials are a separate vulnerability class from application logic flaws. The fix is not deletion from current code — it is rotation of the exposed credential plus integration of a secret manager (HashiCorp Vault, AWS Secrets Manager) and pre-commit scanning to prevent recurrence.

How These Concepts Are Used as an AppSec Engineer

Burp Suite in daily work

Every manual pentest engagement uses Burp Suite. Every time you need to understand what a web application is actually sending and receiving, it is open. The AppSec engineer’s equivalent of a developer’s debugger.

Semgrep in CI/CD

Integrating Semgrep into a company’s CI/CD pipeline, writing rules for company-specific vulnerability patterns, and triaging results is a common AppSec responsibility. The custom rule you built is the portfolio demonstration.

Secrets scanning in CI/CD

Gitleaks runs as a pre-commit hook and a CI gate. TruffleHog runs on a schedule against full git history. When a developer accidentally commits an API key, you have a runbook: alert the developer, rotate the credential immediately (assume it is already compromised), add the pattern to the blocklist. This is day-one operational work at most companies.

Assessment reports

Every engagement produces a report. The format you learned in the Juice Shop assessment is the format used in production. Clear findings, actionable remediations, evidence-backed.

V

Cloud Security Fundamentals

Months 6.25–6.75

Modern AppSec work happens in the cloud. S3 misconfiguration and IAM over-permissioning are among the most common real-world vulnerabilities found by security researchers, and credible threat modeling requires understanding the cloud attack surface an application runs on. This phase covers the AppSec-relevant subset — not Terraform or multi-cloud architecture, but enough to identify misconfigurations in code and architecture reviews, understand SSRF chains that target cloud metadata, and discuss findings with infrastructure teams.

Learning Resources

- **AWS Security CTF: Attacker Path** — flaws.cloud — Six levels covering real-world AWS misconfigurations: S3 bucket exposure, credential leakage, IAM misuse, and SSRF via the EC2 metadata service. Each level explains the vulnerability and its fix.
- **AWS Security CTF: Attacker & Defender** — flaws2.cloud — Companion to flaws.cloud with two separate paths: attacker path (exploit cloud misconfigurations) and defender path (identify and fix the same misconfigurations).
- **AWS Shared Responsibility Model** — aws.amazon.com/compliance/shared-responsibility-model — Official AWS documentation. Defines what AWS secures (infrastructure) versus what the customer secures (IAM, S3 policies, application code, data encryption).
- **AWS IAM Documentation** — docs.aws.amazon.com/iam — Official AWS documentation. The IAM concepts section covers users, roles, groups, policies, and the principle of least privilege.
- **FreeCodeCamp Cloud Security Fundamentals in AWS** — freecodecamp.org — Beginner-friendly guide covering IAM users and policies, S3 bucket configurations, MFA, and the AWS Shared Responsibility Model with hands-on examples.

Cloud Security Fundamentals — Months 6.25–6.75

01. READ: AWS Foundations: Shared Responsibility & IAM: Week 26

- Read the AWS Shared Responsibility Model page completely. Write a one-paragraph summary in your own words: what AWS is responsible for, what you are responsible for, and what the boundary looks like for a web application running on EC2 behind an S3 bucket
- Read the IAM concepts in the AWS IAM documentation: what a user is, what a role is, what a policy is, how they attach, what least privilege means in practice
- Read the FreeCodeCamp Cloud Security Fundamentals in AWS guide — beginner-friendly, hands-on, covers IAM and S3 configuration with real console examples
- **Full notes + note cards:** AWS Shared Responsibility boundary; IAM user vs. role vs. group; what a policy document looks like (JSON structure, Effect/Action/Resource); what least privilege means and why wildcard permissions (s3:*) are a finding

02. LABS: flaws.cloud — Levels 1–6: Week 26–27

- Create a free AWS account if you do not have one. The CTF runs against real AWS infrastructure — you need the AWS CLI installed and configured with a free-tier account
- Complete all six levels. Exploit each level fully before reading the solution
- **For each level:** write standard documentation — what the misconfiguration was, how an attacker exploits it, how to fix it, and what to look for in a code or configuration review to catch it

03. LABS: flaws2.cloud — Attacker & Defender Paths: Week 27

- **Two distinct paths:** attacker (find and exploit misconfigurations) and defender (harden the same environment).
- Complete both paths. The defender path is the AppSec-relevant half — it teaches you what a correctly configured environment looks like, not just what a broken one looks like
- **Key concepts reinforced:** IAM role scoping, S3 bucket policy syntax, EC2 instance metadata v2 (IMDSv2) — the hardened version that prevents credential theft via SSRF

Concepts Mastered

AWS Shared Responsibility Model

AWS secures the physical infrastructure. You secure what runs on it: IAM configurations, S3 policies, application code, data encryption. Misunderstanding this boundary is the root cause of most cloud breaches.

IAM — Identity and Access Management

Users, roles, groups, and policies. A role is what an AWS service or EC2 instance assumes to perform actions. A policy is a JSON document defining what actions are allowed or denied on which resources. Least privilege means a role should have exactly the permissions it needs and nothing more. Over-permissioned roles are the most common IAM finding.

S3 bucket misconfiguration

S3 buckets store data. Public read access means anyone on the internet can read the contents. Authenticated AWS user access means all 300M+ AWS account holders can read it. Both are common misconfigurations. Bucket policies, ACLs, and Block Public Access settings all interact — understanding which one controls what is the skill.

EC2 instance metadata service (IMDS)

169.254.169.254 is accessible from within an EC2 instance and returns IAM credentials, instance ID, and configuration. SSRF against this endpoint gives an attacker the instance's IAM credentials. IMDSv2 requires a token header that prevents simple SSRF exploitation. Whether IMDSv2 is enforced is a standard AppSec checklist item.

Credential exposure in git history

AWS keys committed to a git repository — even if deleted in a later commit — are permanently visible in history. TruffleHog scans history. The fix is to rotate the exposed credential, not just delete it from current code, plus pre-commit scanning to prevent recurrence.

Cloud security code review

Reading an IAM policy JSON, an S3 bucket policy, or a Lambda function configuration and identifying the security problem. This is the AppSec-specific cloud skill — not writing infrastructure, but auditing it.

How These Concepts Are Used as an AppSec Engineer

IAM review in architecture review

When a team presents a new service design, you ask: what IAM role does this service run as? What permissions does that role have? Could an attacker compromise this service and use its IAM role to access something they should not? These questions catch privilege escalation paths before code is written.

SSRF → metadata chain in code review

When you see an endpoint that accepts a URL and fetches it server-side, your first question is now: can this reach 169.254.169.254? Is IMDSv2 enforced on the EC2 instances this service runs on? Is there a URL allowlist? The AWS Security CTF Level 5 scenario is the exact attack chain you are looking for.

S3 policy in configuration review

When reviewing a deployment configuration or an IaC pull request, you check: is Block Public Access enabled at the bucket level? Are there wildcard permissions? Are access logs enabled on buckets containing sensitive data? These are 5-minute checks that catch high-severity findings.

Credentials in code review

When reviewing any code that configures AWS clients, check: are credentials hardcoded? Are they pulled from environment variables (better but still risky) or from a secrets manager or IAM role (correct)?

VI Secure Code Review & Threat Modeling

Months 6.75–7.5

Phase V builds the two highest-leverage AppSec skills — the ones that appear earliest in a job and distinguish engineers from analysts. Secure code review is the daily work of most AppSec engineers at software companies. Threat modeling is how AppSec influence scales beyond individual code reviews to affect system design. The developer perspective that threat modeling requires — understanding how systems are built and where trust boundaries exist — is built through the pre-study and Phases I–IV before this phase begins. OWASP ASVS is introduced here as the verification standard that gives code review a systematic checklist — turning ad-hoc review into structured, repeatable methodology.

Learning Resources

- **OWASP Code Review Guide v2.0** — owasp.org/www-project-code-review-guide — The methodology reference for secure code review.
- **OWASP Application Security Verification Standard (ASVS) v5.0.0** — owasp.org/www-project-application-security-verification-standard — Approximately 350 testable security requirements across 17 chapters covering authentication, access control, input validation, cryptography, API security, and configuration. Level 1 is automatable; Level 2 requires manual testing.
- **"*Threat Modeling: Designing for Security*"** — Adam Shostack, Wiley, 2014 (wiley.com) — The standard text on threat modeling. Ch. 1–5 cover the STRIDE methodology and data flow diagrams.
- **GitHub Security Lab** — securitylab.github.com — Published CVE research on real open-source projects. Covers vulnerability discovery, exploitation mechanics, and coordinated disclosure.
- **OWASP Threat Dragon** — owasp.org/www-project-threat-dragon — Threat modeling tool. Used for the threat modeling project in this phase.

Secure Code Review & Threat Modeling — Months 6.75–7.5

01. READ + PRACTICE: Secure Code Review Methodology: Weeks 28–29

- Read the OWASP Code Review Guide Ch. 1–4 (methodology, code review techniques, vulnerability-specific review guidance)
- Conduct a code review of a real open-source Python web application (Django, Flask, or FastAPI) from GitHub — not WebGoat, but a real project at real scale
- **Apply the systematic methodology:** trace user input, verify authentication, check authorization, audit crypto, check dependencies

02. STUDY: OWASP ASVS — Verification Standard: Week 29

- Read the ASVS v5.0.0 Level 1 and Level 2 requirements. Do not try to memorize — learn the structure and the categories
- Map each ASVS chapter to the vulnerability classes studied in this plan — input validation and output encoding, authentication, access control, and cryptography each get their own chapter, so you can pin any

finding to a specific numbered requirement

- Build a lightweight personal ASVS checklist: 20–30 of the highest-signal Level 2 requirements you would check first in any code review
- This checklist becomes a portfolio artifact and an interview talking point: most entry-level AppSec candidates have not heard of ASVS

03. READ + PROJECT: Threat Modeling: Weeks 29–30

- Read "*Threat Modeling: Designing for Security*" Ch. 1–5
- Build a complete threat model for the API built during pre-study. A system built from scratch end-to-end is the right target: every layer is known, every trust boundary can be reasoned about.
- Use OWASP Threat Dragon for the data flow diagram. Apply STRIDE to each component
- **Produce a threat model document:** DFD, trust boundaries, threat enumeration, risk rating, mitigations

04. PRACTICE: Code Review Under Time Pressure: Week 30

- AppSec interview code review exercises are time-boxed — typically 30–45 minutes to review a snippet and explain findings verbally
- Use GitHub Security Lab's published CVE writeups as answer keys: read the vulnerable code before the writeup, try to identify the vulnerability, then compare
- Do five of these across different vulnerability classes
- **Target:** 30 minutes from cold code to written vulnerability identification

Projects & Writings — Secure Code Review & Threat Modeling

Project: Open-Source Code Review

Documented security review of a real open-source Python application. Findings in standard format, pushed to GitHub. Includes scope, methodology, findings, and remediation recommendations.

Project: Threat Model Document

Full STRIDE threat model of a real-world system architecture. DFD, trust boundaries, threat enumeration, risk ratings, mitigations. Pushed to GitHub. An unusual portfolio artifact — most AppSec candidates have lab writeups but not threat models. A strong target is an LLM-integrated agent that calls tools, forcing STRIDE to cover excessive agency and prompt injection too.

Writing: 'Threat Modeling the Billing System I Built' (1,000–1,500 words)

Use your ActiveCampaign billing microservice as the subject. Walk through what a threat model of that system would have looked like, what STRIDE would have identified, and how the architecture would have been different had AppSec been involved in the design. The most unique piece in the portfolio — no other candidate has this specific experience.

Concepts Mastered

Secure code review as methodology

Not reading code and hoping to notice something bad — systematic input tracing, authentication verification, authorization checking, cryptography auditing. Replicable and teachable.

OWASP ASVS as a verification framework

~350 testable security requirements organized by domain. Level 1 covers basics automatable by scanners. Level 2 requires manual testing and judgment. Using ASVS during code review means you can say ‘this fails ASVS V2.1.1’ rather than ‘this seems wrong’ — a significant credibility upgrade in professional settings.

STRIDE threat modeling

Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege. A threat category for every component of a data flow diagram.

Trust boundaries

The lines in a DFD where data crosses from less-trusted to more-trusted context. Every trust boundary is a potential vulnerability surface.

Time-pressured code review

The AppSec interview format. Cold code, 30–45 minutes, verbal explanation of findings. Practice makes this feel like code review, not an exam.

How These Concepts Are Used as an AppSec Engineer

Code review in practice

When a developer asks you to review a PR, you apply the methodology: trace inputs, check auth, check authz, check crypto, check dependencies. 20 minutes when systematic, hours when ad-hoc.

Threat modeling in architecture review

When a team presents a new microservice design, you build a DFD on the whiteboard, identify trust boundaries, and run STRIDE. One threat model at design time prevents dozens of vulnerability findings after code is written.

VII

CompTIA Security+ Certification

Months 7.75–8.5

CompTIA Security+ is best understood as a resume-screen credential — that is the honest framing. It is required or preferred in a growing number of AppSec job postings, especially at enterprise companies and in regulated industries. It does not demonstrate AppSec competence to a human reviewer. The portfolio does that. But it gets your resume past automated filters at companies that screen for it, and it is achievable in 3–4 weeks with the foundational knowledge built throughout this plan.

Learning Resources

- **CompTIA Security+ Get Certified Get Ahead: SY0-701 Study Guide** — Joe Shelley and Darril Gibson, Certification Experts LLC, 2023 (getcertifiedgetahead.com) — 674 pages covering all SY0-701 exam objectives across eleven chapters with real-world examples and exam topic review sections.
- **Professor Messer Practice Exams** — professormesser.com — Paid practice exams for the Security+ SY0-701 exam. Same format and structure as the actual exam.

CompTIA Security+ Certification Schedule

01. READ + MEMORIZE: Study Guide: Weeks 31–33

- Read CompTIA Security+ Get Certified Get Ahead: SY0-701 Study Guide cover to cover. The exam has five domains: General Security Concepts, Threats/Vulnerabilities/Mitigations, Security Architecture, Security Operations, and Security Program Management & Oversight
- Most technical content from the first three domains will be review from earlier phases. Focus study time on governance, compliance, and program management domains — these are the least familiar material
- **Full notes, then granular cards:** build note cards in book order — one card per concept (title on the front, 1–5 bullets on the back), roughly 160–200 cards, with extra cards where a topic is dense. Prioritize the high-yield memorization sets: ports and protocols, cryptography and PKI, attack types, security control types, and the governance, compliance, and framework material (your least-familiar domain)
- **Memorize (grouped spaced review):** learn cards in groups of seven to full front-to-back recall, fold each group into a growing cumulative review pile, and re-drill the whole pile as it grows (pruning only over-learned cards). Budget real time — at ~200 cards this is a multi-day make-and-memorize effort, not an afterthought

02. PRACTICE: Practice Exams: Weeks 33–34

- Take a full Professor Messer practice exam cold. Score it and identify weak domains
- Study weak domains specifically from the study guide. Retake. Two or three cycles is sufficient

03. EXAM: CompTIA Security+ SY0-701: Week 34

- Take the exam. $392 USD (2026). Valid for three years
- **List on resume as:** CompTIA Security+ SY0-701 (Expires [date])

04. APPLY: Main Application Push: Week 34 onward

- With Security+ now on your resume, intensify the application push that began in Phase IV — widen to more startups and AppSec-adjacent roles
- By now you should have 4–6 published writeups, the Juice Shop assessment PDF, and an application-ready LinkedIn and GitHub
- Continue through interview preparation (Phase VIII); see "Application Strategy" there for targeting, framing, and profile detail

Concepts Mastered

Security+ exam domains

All five SY0-701 domains: General Security Concepts, Threats/Vulnerabilities/Mitigations, Security Architecture, Security Operations, Security Program Management & Oversight

Governance and compliance

Risk management frameworks, regulatory compliance (HIPAA, PCI-DSS, SOC 2), security policy and procedure, security awareness training

Cryptography fundamentals

Symmetric and asymmetric encryption, hashing algorithms, PKI, certificate lifecycle, TLS/SSL configuration

Network security

Firewalls, IDS/IPS, VPN, network segmentation, wireless security

Identity and access management

Authentication protocols, MFA, SSO, directory services, privileged access management

Incident response

IR lifecycle phases, forensics fundamentals, log analysis, chain of custody

How These Concepts Are Used as an AppSec Engineer

ATS filter

Security+ signals baseline security literacy to automated resume screening at enterprise companies and regulated industries. It does not substitute for portfolio work in human interviews.

Governance vocabulary

AppSec engineers work with compliance teams. Knowing what PCI-DSS requires for web applications, what HIPAA means for data handling, and how risk frameworks are structured makes cross-team conversations productive.

Common interview ground

Cryptography, network security, and authentication fundamentals from Security+ appear in entry-level AppSec interviews. The certification confirms baseline literacy before the technical AppSec questions start.

VIII

Interview Preparation & Interviewing

Months 8.75–9.25

AppSec interviews test security knowledge, adversarial reasoning, and communication — not algorithmic problem solving. Prep splits into two tracks. Track A is pure memorization and rehearsal — behavioral answers, past-job stories, and the foundational AppSec question-and-answer set — delivered aloud until it is natural. Track B is different: the live, watched formats (secure code review out loud, light scripting) are a performance skill, and they fail on nerves, not knowledge, so Track B is trained by desensitization — timed, watched reps at volume, spread across months and started early, not crammed here. Both tracks, and how each maps to the interview formats, are covered below.

Track A: Behavioral Question Rehearsals

Write out full answers to the stock behavioral questions, memorize them, then rehearse out loud until delivery is natural rather than recited. Cover past experience and previous jobs, the typical ‘toughest challenge you faced,’ ‘a time you disagreed with a teammate,’ and ‘why this transition into security.’ Keep a tight version and an expanded version of each so you can scale the answer to the interviewer’s follow-ups.

- Rehearse the AppSec-specific behavioral prompts too: handling a developer who pushes back on a finding, prioritizing a backlog of vulnerabilities, and explaining risk to a non-technical stakeholder.

Track A: Technical Note Cards

Organize note cards by topic — Python, software development (general), SQL and database normalization, and the security domains (OWASP Top 10, the individual vulnerability classes, and defensive patterns). The cards are the most load-bearing prep tool, and the review method matters as much as the cards themselves.

- **Foundational AppSec questions:** alongside cards drawn from your own study, seed a deck from a curated interview-question bank such as jassics/security-interview-questions so you rehearse the questions that actually get asked, in your own words.

Review method: study in sub-groups of 7 cards. Review each sub-group 5 times, until those 7 are memorized. Then combine that sub-group into your running ‘reviewed’ pile and review the entire reviewed pile again. Anything missed on that pass, re-review a couple more times. Repeat the cycle — 5 passes on the next sub-group of 7, combine, then a full reviewed-pile pass — until every card for the topic is covered. As upkeep, read through the different reviewed groups periodically so they stay locked in.

Track B: Technical Programming Practice

The hands-on formats — secure code review, light scripting, and system design — reward deliberate practice the same way an algorithms candidate grinds patterns. Do not just complete exercises; internalize the patterns, why they work, and when each applies.

- **Secure code review:** practice the read → identify the vulnerability class → explain the exploitation → recommend the fix loop until it is automatic, and do it out loud — interviews require spoken fluency. Use

GitHub Security Lab's published CVE writeups as a source of real code for cold practice; aim for five cold reviews per week during this phase.

- **Light scripting:** some companies include a CoderPad session — write a small Python script to parse a log file, find a bug in a function, or automate a security check. Not LeetCode. Build fluency at writing correct Python quickly without an IDE; a book like "*Impractical Python Projects*" (2019) is good for varied, self-contained practice. Your development background carries most of this.
- **System design with security focus:** practice designing and critiquing architectures for security — design a secure authentication system, or review a proposed microservice design for trust boundaries and STRIDE threats. Rehearse explaining the design verbally; your developer background is the advantage here.

Sample cold-review snippets to drill until the class is obvious on sight:

- **Python:** a web endpoint accepting user input without sanitization — identify the injection class
- **JavaScript:** a React component rendering props with dangerouslySetInnerHTML — identify the XSS vector
- **Python:** a JWT decoded with algorithm='none' accepted — identify the authentication bypass
- **Python:** a user ID taken from a request parameter without an authorization check — identify the IDOR
- **Python:** a URL parameter fetched server-side without validation — identify the SSRF

Track B: Desensitizing the Live Formats (Start Early)

The point of Track B is exposure, not learning — make the watched, timed format so routine that it stops triggering the freeze. Do not save this for Month 8.75: begin as soon as you can read vulnerable code (Phase III), keep it running through every later phase, and have mock interviews underway well before your first real interview.

- **Graded exposure ladder:** climb one rung at a time — solo and untimed, then solo on a timer, then recorded and played back, then watched live by a mentor or peer, then watched by a stranger. Move up only once the current rung stops spiking your anxiety.
- **Rehearse the recovery, not just the solve:** practice narrating your thinking out loud, and drill the explicit 'I am not certain yet — here is how I would approach it' move, so a blank moment becomes a spoken plan instead of a silent lock-up. That recovery line is the single most valuable thing to have automatic.
- **Mock cadence:** a couple of watched or timed reps each week from Phase III onward, ramping to a full mock (behavioral plus code review) roughly monthly once applications begin at Month 4. Never let a real interview be the first time you perform the format cold.
- **Where to run mocks:** MentorCruise for AppSec-specific mentor mocks (code review plus calibration); Exponent's Security Engineering track for structured security mocks; and interviewing.io for cheap, high-volume behavioral and coding-under-observation reps (general, not AppSec-specific — use it for the desensitization volume).

Interview Preparation

Each interview format maps to one of the methods above:

- **Behavioral** — your pre-written answers and rehearsals.

- **Conceptual security questions** — note-card review: OWASP concepts, vulnerability explanations, and defensive recommendations, no code required.
- **Secure code review exercise** — the secure-code-review routine. The most common AppSec technical screen: a code snippet in Python, JavaScript, or Java with embedded vulnerabilities, 30–45 minutes, verbal explanation of findings required.
- **System design with security focus** — the system-design practice; your developer background is the advantage.
- **Light scripting** — your Python practice. A short CoderPad task (parse a log, fix a function, automate a check), not LeetCode.

Framing Your Background

- **The narrative:** Five years of full-stack software engineering. Deliberate transition into application security since June 2026. Chose foundational independent study over bootcamp shortcuts.
- **The gap:** ‘I have been in a full-time transition into application security since June 2026. Here is what that looks like.’ Share specific projects. The portfolio is the evidence.
- **The advantage:** ‘I spent five years building the systems AppSec engineers defend. I know why developers write vulnerable code — because I wrote it too.’
- **Target roles:** Application security engineer, product security engineer, security-focused software engineer, DevSecOps engineer. Do not filter narrowly on job title.

Application Strategy

- **Start applying at Month 4:** Security-conscious startups are viable targets from Month 4 with Phases I–III complete, two published writeups, and PortSwigger Academy progress visible. Do not wait for curriculum completion.
- **Pin your best AppSec work:** set your GitHub pinned repositories to the key new artifacts — the Juice Shop assessment report, your custom Semgrep rule, and the threat model — so a reviewer sees application-security work first, not the older web-dev projects.
- **Make LinkedIn application-ready:** Security+ and, once passed, BSCP listed under licenses & certifications; portfolio projects and published writeups linked; and the About section carrying the AppSec transition narrative.
- **Frame the gap explicitly:** Cover letters should state upfront: ‘After five years of full-stack engineering, I made a deliberate transition into application security in June 2026.’ This plan is the evidence.
- **Target AppSec-adjacent roles too:** Security engineer, product security engineer, DevSecOps, security-focused software engineer — all overlap substantially with AppSec. Do not filter too narrowly on title.
- **Your full-stack background is the pitch:** ‘I spent five years building the attack surface. Now I am learning to break it — so I can help teams build more securely.’

IX Expected First Days on the Job

Months 9.0+

You will know the vulnerability classes better than many colleagues who arrived through non-engineering paths. You will be slower on internal tooling, company-specific security policies, and the organizational dynamics of working with development teams at varying security maturity. That gap closes in weeks, not months. The engineering history is the relevant benchmark: hired at 14 weeks of web dev self-study, productive within the first weeks of employment.

First Two Weeks

Read the existing security documentation the way you read a codebase — understand what is there before suggesting what should change. Ask about the current vulnerability backlog before proposing new tooling. Understand what development teams build and how they ship before inserting yourself into their process.

- What is the current SAST setup, if any? What do developers receive as output?
- What does the existing pentest methodology look like? How are findings tracked?
- Who are the key development team contacts? What are their current security concerns?
- What does the incident response process look like? Has it been tested?

> The question to ask in the first week is not ‘what should we add?’ It is ‘what is already here, and why is it the way it is?’ The answer tells you everything about the political and organizational constraints you are working within.

Your Immediate Differentiators

Developer empathy

You understand why developers write vulnerable code because you wrote it too. This makes you a better communicator than security professionals who have never shipped production software.

API security depth

REST API security is the dominant AppSec domain in 2026. Hands-on API security knowledge built throughout this plan means arriving on day one prepared for the work.

Automation instinct

Security teams full of former analysts often under-automate. Your engineering background means Semgrep rule development, CI/CD integration, and tooling scripting come naturally.

Full-stack deployment

You can own the deployment of security tooling from writing the Semgrep rule to integrating it into GitHub Actions to triaging the results. Most AppSec engineers need help with the deployment layer.

The Gap and How Long It Takes to Close

The distance between ‘knows the vulnerability classes and tooling’ and ‘productive AppSec team member who moves fast with developers’ is 4–8 weeks for someone with your foundation and learning velocity. The developer background compresses this timeline — you already speak the language of the teams you will be working with. You will not be a liability. You will be a junior on a clearly steep upward curve — which is what good security teams want to hire.

Appendix

Books & Learning Resources

Title / Source	Author / Year	Phase	Notes
"*Alice and Bob Learn Application Security*"	Tanya Janca, 2020	II–III	wiley.com. The most accessible AppSec entry point in print. SDLC security, threat modeling intro, developer collaboration. Appears on every AppSec practitioner reading list.
"*Real-World Bug Hunting*"	Yaworski, 2019	I–III	nostarch.com. Most accessible intro to web vulns. Organized by vuln class with real bug bounty reports.
"*The Web Application Hacker's Handbook*"	Stuttard & Pinto, 2nd ed. 2011	II–III	Dafydd Stuttard and Marcus Pinto, Wiley, 2011 (wiley.com). Ch. 1–3 cover fundamentals. Ch. 6 authentication, Ch. 7 session management, Ch. 8 access controls.
"*Hacking APIs*"	Corey Ball, 2022	III–IV	nostarch.com. REST API security testing. Direct overlap with your API background.
"*Threat Modeling: Designing for Security*"	Shostack, 2014	V	wiley.com. Standard text. Ch. 1–5 are primary content.
OWASP WebGoat + OWASP Juice Shop	—	I / IV	github.com/WebGoat/WebGoat and github.com/juice-shop. Run via Docker. Deliberately vulnerable apps for Phase I and IV respectively.
TryHackMe	—	I–II	tryhackme.com. Browser-based guided rooms. Web Fundamentals and Jr Penetration Tester paths.
OWASP Threat Dragon	—	VI	owasp.org/www-project-threat-dragon. DFD and threat modeling tool.

Certifications

Certification	Provider	Phase	Cost	Notes
Security+ SY0-701	CompTIA	VI	$392	ATS filter-passer. Required for many enterprise and regulated industry postings.

Burp Suite Certified Practitioner	PortSwigger	IV	—	Practical exam. Highly respected in AppSec community. Stronger technical signal than Sec+.
GWEB	GIAC	Post-job	~$999	GIAC Web Application Defender. Specialized, respected, expensive. Realistic after first role.
CSSLP	ISC2	Post-job	~$599	Certified Secure Software Lifecycle Professional. SDLC-focused. Useful for senior roles.

Practical Use Per Phase

Phase I — AppSec Intro

Use Claude to explain why a WebGoat exploit works at the code level. Ask it to describe the difference between SQL injection and command injection in plain English. Have it generate sample vulnerable Python code for practice — and then explain what makes it vulnerable.

Phase II — Web & Network Foundations

Use Claude to explain HTTP security headers step by step: what does SameSite=Strict actually prevent, mechanically? Have it walk through an OAuth 2.0 flow and identify where common vulnerabilities occur. Ask it to explain the difference between authentication and authorization until it clicks.

Phase III — Core Vulnerability Classes

Use Claude to generate additional practice examples for vulnerability classes where PortSwigger labs are thin. Have it produce a vulnerable Python function and explain exactly which line introduces the vulnerability and why. Ask it to explain why a particular XSS payload bypasses a specific filter.

Phase IV — Security Testing & Tooling

Use Claude to review your custom Semgrep rule and identify edge cases it misses. Have it explain a Bandit finding ID you have not seen before (e.g. B324 hashlib insecure). Have it explain Burp Suite Repeater output when a response is unexpected. Ask it to help write the executive summary section of the Juice Shop report.

Phase V — Cloud Security Fundamentals

Use Claude to explain what an IAM policy JSON document is doing in plain English. Ask it to identify the security problem in a sample bucket policy you paste in. Have it explain the difference between IMDSv1 and IMDSv2 and why the distinction matters for SSRF. Ask it to help write up a cloud security finding in professional report format.

Phase V — Code Review & Threat Modeling

Use Claude as a review partner: paste a code snippet and ask it to identify vulnerabilities from the perspective of an AppSec engineer. Have it critique your threat model document: what threats did STRIDE miss? What mitigations are insufficient?

Phase VI — Sec+ Certification

Use Claude as a study partner for practice questions. Give it a domain you are weak on and ask it to quiz you. Have it explain compliance frameworks (NIST CSF, ISO 27001) in plain English — the governance domains are the new material.

Phase VII — Sec+ & Interviews

Use Claude as a technical interviewer. Give it a vulnerable code snippet and ask it to evaluate your explanation as if it were an AppSec hiring manager. Have it generate the strongest possible objections to your vulnerability findings — surface the gaps before an interview does.

Philadelphia AppSec Community

Security networking operates differently from general tech networking. The community is smaller, more tight-knit, and more willing to help people who show genuine curiosity. Attending events before you are employed builds relationships that become referrals. Show up, ask good questions, and bring something to demonstrate — even a completed PortSwigger lab that taught you something surprising.

Core Groups

OWASP Philadelphia Chapter

owasp.org/www-chapter-philadelphia · Free

The most directly relevant group for AppSec networking in Philadelphia. OWASP (Open Web Application Security Project) is the professional organization for web application security globally. The Philadelphia chapter holds regular meetings with technical presentations on vulnerability classes, tooling, and real-world AppSec work. The people in this room are AppSec practitioners at companies across the region — exactly the referral network you are building toward.

DC215 — Philadelphia DEF CON Group

dc215.org · Free

DEF CON Groups are local offshoots of DEF CON, the largest hacker conference in the world. DC215 holds regular meetups with technical talks, CTF practice, and informal knowledge sharing. Builder-focused and practitioner-dense. The format rewards people who bring something to show. Attend when Phase III work (vulnerability classes) is underway.

BSides Philadelphia

bsidesphilly.org · Annual · Low cost

Community-organized security conference with talks across offensive, defensive, and AppSec tracks. Affordable, full-day, attended by local practitioners from the companies you will apply to. Volunteer if possible — it is how you get face time with organizers and speakers.

Philly 2600

philly2600.net · First Friday monthly · Free

Philadelphia's oldest hacker meetup — running since the early 1990s and one of the first 2600 meetings ever listed in 2600 Magazine. Meets the first Friday of every month at Iffy Books, 404 S 20th St, starting at 6pm. No set agenda, no presentations, no gatekeeping — a casual hangout where people bring things to hack on or show off. Open to anyone. Around 8pm the group typically moves to a nearby bar. The lowest-pressure entry point into the Philly security community and a good place to show up early in the plan before you have polished work to present.

PhillySec

meetup.com · Second Wednesday monthly · Free

Monthly security meetup, second Wednesday of each month. More structured than Philly 2600 — presentations and technical discussions rather than open hangout format. Good mid-plan target once Phase II or III is underway and you have something to contribute to conversations.

Ex Machina Parlor

319 N 11th St, Room 501 · Philadelphia

A hackerspace specifically aimed at people interested in cybersecurity — formerly known as Trashpanda Village and SecShell. Workshop and collaborative hacking space rather than a meetup. Worth knowing it exists as a physical space to work alongside practitioners. Check their calendar for events and open nights.

Conferences

- **OWASP Global AppSec — Washington DC** (owasp.org/events · Annual · November) — The flagship AppSec-specific conference in the US. 800+ practitioners, six tracks including builder/developer, breaker, defender, and OWASP projects. Washington DC is a short trip from Philadelphia. This is the single most targeted conference in the plan — the audience is exactly the community you are entering and the talks directly map to the skills in every phase. Attend once Phase IV or V is complete.
- **OWASP BASC — Boston** (owasp.org · Annual · Fall) — The OWASP Boston Application Security Conference. Regional, practitioner-dense, focused on AppSec engineering and secure development. Closer and lower-cost than the DC Global AppSec. Strong emphasis on hands-on content and OWASP standards — the audience overlaps heavily with who you will work alongside in the role.
- **Thotcon — Chicago** (thotcon.org · Annual · May) — A mid-sized hacker conference: more intimate than DEF CON, technically dense, practitioner-focused. Worth targeting once Phase III or IV work is underway and you have something real to talk about.
- **JawnCon — Philadelphia** (jawncon.org · Annual) — Philadelphia's own security and technology conference. Covers a wide range of security and tech topics with a deliberately approachable, community feel. Worth attending as a local conference where you will run into the same practitioners from OWASP and DC215.
- **DEF CON — Las Vegas** (defcon.org · Annual · August) — DEF CON 34 runs August 6–9, 2026 at the Las Vegas Convention Center. The flagship hacker conference. Within DEF CON, the AppSec Village (appsecvillage.com) is a 100% volunteer-run space specifically for application security — attacker-minded talks, live demos, hands-on workshops, a Fix-the-Flag CTF that rewards remediating vulnerabilities, and rapid tool demos at the Arsenal. The AppSec Village is the most directly relevant part of DEF CON for this career path. Attend once Phase IV or V is complete.